ДИСЦИПЛИНА «Разработка программных средств поиска и анализа киберугроз»

ИНСТИТУТ «Институт кибербезопасности и цифровых технологий»

КАФЕДРА КБ-2 «Информационно-аналитические системы кибербезопасности»

ВИД УЧЕБНОГО МАТЕРИАЛА Лекция

ПРЕПОДАВАТЕЛЬ Новиков Евгений Иванович

СЕМЕСТР 9 семестр 2026-2027 учебный год

ЛЕКЦИЯ 4. ОЧИСТКА ДАННЫХ В КИБЕРБЕЗОПАСНОСТИ ОШИБКИ, ДУБЛИКАТЫ, ПРОПУСКИ

План лекции:

1. Выявление и обработка ошибок в данных.
2. Выявление и обработка дубликатов.
3. Выявление и обработка пропущенных значений.

In [2]:
```
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%config InlineBackend.figure_format = 'retina'
pd.set_option('display.max_colwidth', None)
import scipy.stats as sts
```

Очистка данных в кибербезопасности: ошибки, дубликаты, пропуски

Гавное правило машинного обучения, которое в кибербезопасности работает жестче, чем в других сферах: **«Garbage In, Garbage Out»** — «Мусор на входе, мусор на выходе».

В классическом ML ошибка в данных означает, что ваша модель немного хуже предскажет цену дома или рейтинг фильма. В кибербезопасности «мусор на входе» означает две вещи. Первое: модель пропустит целевую атаку (False Negative), и вы даже не узнаете об этом. Второе: модель начнет генерировать тысячи ложных срабатываний (False Positive), что приведет к «алертной усталости» (alert fatigue) аналитиков SOC, и они просто перестанут реагировать на реальные инциденты. Поэтому качество данных — это не просто метрика, это вопрос выживания.

На практике мы редко получаем красивые, чистые CSV-файлы, как в учебниках.

Реальные данные это:

- **сетевые потоки (NetFlow, IPFIX)**: IP-адреса, порты, флаги TCP, объемы переданных байт.
- **логи событий (SIEM, EDR)**: события безопасности Windows, Syslog, логи антивирусов, которые часто представляют собой неструктурированный текстовый хаос.
- **дампы трафика (PCAP)**: сырые пакеты, где нужно извлекать признаки из заголовков.
- **характеристики файлов**: PE-заголовки, энтропия байт-кода, импортные таблицы.

Все эти источники генерируют терабайты данных. И эти данные *грязные*. Сбои в парсерах, потеря пакетов в сети, намеренное искажение данных злоумышленником, странные форматы времени — это повседневная реальность.

Как превратить эти "сырые" данные в математические признаки (features), которые поймет алгоритм? Для этого нужен **конвейер предобработки данных (Data Preprocessing Pipeline)**.

В ходе лекций и практических занятий мы разберем, как находить и чинить **ошибки**, как бороться с **дубликатами**, которые ухудшают модели. Научимся правильно работать с **пропущенными значениями**. Мы поговорим о том, как **масштабировать** признаки, чтобы гигантские выбросы не сломали метрики, как применять **нелинейные преобразования** для выравнивания распределений, и, наконец, как **кодировать категориальные признаки** вроде IP-адресов и доменных имен, чтобы не утонуть в «проклятии размерности».

Наша цель — не просто зачистить данные, а сохранить в них криминалистическую и аналитическую ценность.

1. Выявление и обработка ошибок в данных

Ошибка в данных — это не всегда «случайный сбой». В кибербезопасности мы сталкиваемся с тремя принципиально разными природами ошибок, и каждая требует своего подхода.

Технические ошибки. Сломался парсер syslog-агента, и вместо JSON мы получили битую строку. Сетевой сенсор потерял пакет, и временная метка получила дату 1970 год. Это шум инфраструктуры.

Логические ошибки. Данные технически валидны, но семантически невозможны. Например, размер TCP-пакета равен минус пяти байтам. Или порт назначения — 99999, хотя максимальный порт — 65535. Или время начала сессии позже времени её завершения. Такие ошибки часто возникают из-за багов в самом агенте сбора телеметрии.

Намеренные искажения. Злоумышленник знает, что вы собираете логи, и специально их модифицирует: подменяет IP-адреса, искажает временные метки, добавляет мусорные события, чтобы «ослепить» вашу модель. Это уже не ошибка — это атака на сам пайплайн данных.

Рассмотрим технологии обработки ошибок на примере датасет, который моделирует реальные сетевые потоки. Представьте, что мы получили его от сетевого сканера после суток работы.

In [ ]:
```python
# @title
from datetime import datetime
import ipaddress

# Создаем учебный датасет сетевых потоков
# Имитируем данные, которые мог бы собрать NetFlow-сенсор
data = {
    'timestamp': [
        '2026-07-14 10:15:23',      # нормальная метка
        '2026-07-14 10:15:25',      # нормальная
        '1970-01-01 00:00:00',      # ОШИБКА: сброс времени (сбой сенсора)
        '2026-07-14 10:15:30',      # нормальная
        '2026-07-14 10:15:32',      # нормальная
        'not-a-date',               # ОШИБКА: битый парсер
        '2026-07-14 10:15:40',      # нормальная
        '2025-01-01 00:00:00',      # ОШИБКА: метка из прошлого (рассинхронизация)
    ],
    'src_ip': [
        '192.168.1.10',
        '10.0.0.5',
        '192.168.1.15',
        '999.999.999.999',          # ОШИБКА: невалидный IP
        '172.16.0.22',
        '192.168.1.10',
        'abc.def.ghi.jkl',          # ОШИБКА: мусор в поле IP
```

```
        '10.0.0.5',
    ],
    'dst_port': [
        443,
        80,
        22,
        99999,                      # ОШИБКА: порт > 65535
        8080,
        443,
        -1,                         # ОШИБКА: отрицательный порт
        53,
    ],
    'bytes_sent': [
        1500,
        3200,
        450,
        12000,
        -500,                       # ОШИБКА: отрицательный объем
        800,
        0,
        2100,
    ],
    'protocol': [
        'TCP', 'TCP', 'TCP', 'TCP',
        'UDP',                      # ОШИБКА: порт 8080 работает по протоколу TCP
        'TCP', 'UDP', 'TCP'
    ]
}

df = pd.DataFrame(data)
df
```

Out[ ]:

	timestamp	src_ip	dst_port	bytes_sent	protocol
0	2026-07-14 10:15:23	192.168.1.10	443	1500	TCP
1	2026-07-14 10:15:25	10.0.0.5	80	3200	TCP
2	1970-01-01 00:00:00	192.168.1.15	22	450	TCP
3	2026-07-14 10:15:30	999.999.999.999	99999	12000	TCP
4	2026-07-14 10:15:32	172.16.0.22	8080	-500	UDP
5	not-a-date	192.168.1.10	443	800	TCP
6	2026-07-14 10:15:40	abc.def.ghi.jkl	-1	0	UDP
7	2025-01-01 00:00:00	10.0.0.5	53	2100	TCP

Видно, что в данных присутствуют ошибки.

Для обнаружения ошибок в реальных датасета, содержащих миллионы записей, безусловно нужна автоматизация.

Рассмотрим пример технологии выявления ошибок.

Уровень 1. Валидация формата данных.

Первое, что мы проверяем — соответствует ли поле ожидаемому формату.

Для IP-адресов можно использовать стандартный модуль `ipaddress`, для дат — `pd.to_datetime`.

Проанализируем форматы данных всех признаков.

In [ ]:
```
df.dtypes
```

Out[ ]:

	0
timestamp	object
src_ip	object
dst_port	int64
bytes_sent	int64
protocol	object

dtype: object

Обратите внимание на параметр `errors='coerce'`. Он превращает все невалидные даты в `NaT` (Not a Time), и мы сразу видим, какие строки содержат ошибки. В реальном пайплайне такие строки мы либо отправляем на ручную разборку аналитику, либо отбрасываем, либо — что чаще — помечаем флагом и отправляем в отдельный датасет для расследования инцидентов самого сенсора.

In [ ]:
```
# Проверка валидности IP-адресов
def is_valid_ip(ip):
    try:
        ipaddress.ip_address(str(ip))
        return True
    except ValueError:
        return False

df['ip_valid'] = df['src_ip'].apply(is_valid_ip)

# Проверка временных меток
df['ts_parsed'] = pd.to_datetime(df['timestamp'], errors='coerce')
df['ts_valid'] = df['ts_parsed'].notna()

display(df[['src_ip', 'ip_valid']], df[['timestamp', 'ts_parsed', 'ts_valid']])
```

	src_ip	ip_valid
0	192.168.1.10	True
1	10.0.0.5	True
2	192.168.1.15	True
3	999.999.999.999	False
4	172.16.0.22	True
5	192.168.1.10	True
6	abc.def.ghi.jkl	False
7	10.0.0.5	True

	timestamp	ts_parsed	ts_valid
0	2026-07-14 10:15:23	2026-07-14 10:15:23	True
1	2026-07-14 10:15:25	2026-07-14 10:15:25	True
2	1970-01-01 00:00:00	1970-01-01 00:00:00	True
3	2026-07-14 10:15:30	2026-07-14 10:15:30	True
4	2026-07-14 10:15:32	2026-07-14 10:15:32	True
5	not-a-date	NaT	False
6	2026-07-14 10:15:40	2026-07-14 10:15:40	True
7	2025-01-01 00:00:00	2025-01-01 00:00:00	True

Уровень 2. Валидация диапазона.

Второй уровень — проверка, попадают ли числовые значения в физически возможные диапазоны. В кибербезопасности эти диапазоны обычно диктуются протоколами и стандартами.

In [ ]:
```
# Проверка диапазонов
df['port_valid'] = df['dst_port'].between(0, 65535)
df['bytes_valid'] = df['bytes_sent'] >= 0

# Временная логика: метка не должна быть "из прошлого"
# (например, не старше 1 года от текущей даты)
min_allowed_date = pd.Timestamp('2025-07-14')
df['ts_logic_valid'] = df['ts_parsed'] >= min_allowed_date

df[['src_ip', 'dst_port', 'bytes_sent', 'timestamp',
    'ip_valid', 'port_valid', 'bytes_valid', 'ts_logic_valid']]
```

Out[ ]:

	src_ip	dst_port	bytes_sent	timestamp	ip_valid	port_valid	bytes_valid	ts_logic
0	192.168.1.10	443	1500	2026-07-14 10:15:23	True	True	True	
1	10.0.0.5	80	3200	2026-07-14 10:15:25	True	True	True	
2	192.168.1.15	22	450	1970-01-01 00:00:00	True	True	True	
3	999.999.999.999	99999	12000	2026-07-14 10:15:30	False	False	True	
4	172.16.0.22	8080	-500	2026-07-14 10:15:32	True	True	False	
5	192.168.1.10	443	800	not-a-date	True	True	True	
6	abc.def.ghi.jkl	-1	0	2026-07-14 10:15:40	False	False	True	
7	10.0.0.5	53	2100	2025-01-01 00:00:00	True	True	True	

Выявлены классические ошибки:

- отрицательные байты;
- порты за пределами 16-битного диапазона;
- временные метки из «доисторического» прошлого.

Важный момент: границы диапазонов должны быть задокументированы. Если в будующем появится новый протокол с 32-битными портами, созданная валидация должна быть легко обновляема. Рекомендуется хранить такие правила в конфигурационных файлах, а не в скриптах.

Уровень 3. Логическая валидация.

Третий уровень — это проверка бизнес-логики. Например, если у нас есть поля `start_time` и `end_time`, то `end_time` не может быть раньше `start_time`. Если у нас есть `bytes_sent` и `bytes_received`, то их сумма должна соответствовать `total_bytes`.

Рассмотрим одну простую, но очень частую ошибку — несоответствие протокола и порта.

В нашем примере порту 8080 соответствует протокол UDP, хотя этот порт используется протоколом TCP.

Для выявления подобных несоответствий можно использовать классические механизмы создания словарей.

In [ ]:
```python
import pandas as pd

valid_pairs = {
        80: "TCP",
        443: "TCP",
        8080: "TCP",
        22: "TCP",
        53: "TCP"
}

def check_port_protocol_pair(port, protocol, port_valid):
    """
    Возвращает True, если:
      - порт валиден (port_valid == True)
      - и пара (порт, протокол) есть в словаре эталонов.
    В остальных случаях — False.
    """
    # Если порт невалиден по предыдущим проверкам — сразу False
    if not port_valid:
        return False

    expected_proto = valid_pairs.get(port, None)
    if expected_proto is None:
        # Порт валиден, но не входит в наш список «нормальных» пар — считаем аномалие
        return False

    return expected_proto.upper() == protocol

# Применяем функцию построчно
df['port_proto_valid'] = df.apply(
```

```
    lambda row: check_port_protocol_pair(row['dst_port'], row['protocol'], row['port_
    axis=1
)
```

In [ ]:
```
df
```

Out[ ]:

	timestamp	src_ip	dst_port	bytes_sent	protocol	ip_valid	ts_parsed	ts_valid	p
0	2026-07-14 10:15:23	192.168.1.10	443	1500	TCP	True	2026-07-14 10:15:23	True	
1	2026-07-14 10:15:25	10.0.0.5	80	3200	TCP	True	2026-07-14 10:15:25	True	
2	1970-01-01 00:00:00	192.168.1.15	22	450	TCP	True	1970-01-01 00:00:00	True	
3	2026-07-14 10:15:30	999.999.999.999	99999	12000	TCP	False	2026-07-14 10:15:30	True	
4	2026-07-14 10:15:32	172.16.0.22	8080	-500	UDP	True	2026-07-14 10:15:32	True	
5	not-a-date	192.168.1.10	443	800	TCP	True	NaT	False	
6	2026-07-14 10:15:40	abc.def.ghi.jkl	-1	0	UDP	False	2026-07-14 10:15:40	True	
7	2025-01-01 00:00:00	10.0.0.5	53	2100	TCP	True	2025-01-01 00:00:00	True	

Отобразим только валидные записи таблицы.

In [ ]:
```
validation_cols = ['ip_valid', 'ts_valid', 'port_valid',
                   'bytes_valid', 'ts_logic_valid', 'port_proto_valid']
df['is_clean'] = df[validation_cols].all(axis=1)

df[['src_ip', 'dst_port', 'protocol', 'is_clean']]
```

Out[ ]:

	src_ip	dst_port	protocol	is_clean
0	192.168.1.10	443	TCP	True
1	10.0.0.5	80	TCP	True
2	192.168.1.15	22	TCP	False
3	999.999.999.999	99999	TCP	False
4	172.16.0.22	8080	UDP	False
5	192.168.1.10	443	TCP	False
6	abc.def.ghi.jkl	-1	UDP	False
7	10.0.0.5	53	TCP	False

В реальных SIEM-системах коэффициент отброса на этапе валидации формата может достигать 5–15%. Это не потеря данных — это фильтрация шума.

Так же необходимо понимать, что все эти «ошибки» в нашем датасете могут быть не случайностью. Злоумышленник, который уже находится внутри периметра, может специально генерировать битые логи, чтобы:

- сломать парсер и вызвать падение агента сбора данных;
- зашумить выборку, чтобы его вредоносная активность растворилась в потоке аномалий;
- спровоцировать отброс легитимных событий, которые модель посчитает «ошибочными».

Это называется data poisoning attack (атака отравлением данных), и это реальная угроза для ML-моделей в кибербезопасности.

На практике применяются различные подходы для выявления подобных атак:

- контроль целостности логов. Использование криптографических хешей (например, SHA-256) для цепочек логов. Если хеш не сходится — лог был изменен;
- мониторинг статистик в реальном времени. Если вдруг процент «битых» логов с одного хоста резко вырос — это не сбой, это индикатор компрометации (IoC);
- правило «подозрительное — значит интересное». В классическом ML мы отбрасываем аномальные строки. В кибербезопасности мы сохраняем их в отдельный датасет и анализируем. Часто именно «битые» логи — это следы работы эксплойта.

2. Выявление и обработка дубликатов

Дубликаты - это одинаковые данные (записи). Разделяются на полные и частичные. Дубликаты приводят к избыточности, увеличивают объем выборки, при этом не повышая информативность данных.

В классическом Data Science дубликат — это просто лишняя строка, которую нужно удалить, чтобы не тратить память. Но в задачах выявления киберугроз всё сложнее.

2.1 Дубликаты в структурированных данных

Дубликат в структурированных данных может быть:

- техническим артефактом (повторная запись события сенсором, ошибка конвейера сбора);
- частью атаки (например, при сканировании портов или DDoS один и тот же паттерн повторяется много раз — и это именно то, что мы хотим детектировать);
- проявлением легитимной активности (автоматические опросы сервисов, heartbeat-сообщения).

Поэтому в кибербезопасности стратегия «удалить все дубликаты без разбора» опасна: можно стереть сигнатуру атаки. Наша цель — не просто «очистить» данные, а выделить **технические дубликаты** (ошибки сбора) и **семантически значимые дубликаты** (паттерны угроз).

Причины появления дубликатов (типичные сценарии в ИБ):

1. Дублирование на уровне сенсоров. Один и тот же сетевой поток фиксируется сразу несколькими сенсорами (IDS, межсетевой экран, сетевой коллектор). В результате одна сессия попадает в лог 2–3 раза. Протокол TCP надежный. Если пакет потерялся, он отправляется снова. Сетевой сенсор может записать и первый, и второй пакет. Для модели это будет выглядеть как два разных события, хотя физически это одна передача данных.
2. Агенты безопасности (EDR, endpoint-агенты) работают по принципу «гарантированной доставки». Если SIEM-сервер не ответил «OK» на отправку лога, агент отправит его снова. В итоге в вашей базе одно и то же событие Windows Event Log может лежать в трех экземплярах.
3. Агрегация с перекрытием окон. При скользящей агрегации трафика по временным окнам (например, 1 минута) одни и те же события на границах окон могут попасть в два соседних окна.
4. Ошибки парсинга и нормализации. Один сырой лог может породить несколько записей из-за неверной логики разбиения строк.
5. Атака как источник повторов. Злоумышленник сканирует порты, перебирает пароли (брутфорс) или делает тысячи одинаковых HTTP-запросов. Технически это не дубликаты — у них разные временные метки. Но семантически — это одно и то же повторяющееся действие. И если мы просто оставим эти строки как есть, мы исказим статистику.

Ключевое правило: если дубликат вызван ошибкой сбора — удаляем; если это часть атаки или легитимного паттерна — сохраняем, но можем агрегировать или маркировать.

Рассмотрим технологии обработки дубликатов на следующем наборе данных.

In [ ]:
```
# @title
data = {
    'timestamp': pd.to_datetime([
        '2026-07-14 10:00:01', '2026-07-14 10:00:05', '2026-07-14 10:00:05',
        '2026-07-14 10:00:10', '2026-07-14 10:00:15', '2026-07-14 10:00:20',
        '2026-07-14 10:00:25', '2026-07-14 10:00:30', '2026-07-14 10:00:35'
    ]),
```

```
    'src_ip': [
        '192.168.1.10', '10.0.0.5', '10.0.0.5',
        '172.16.0.22', '192.168.1.10', '10.0.0.99',
        '10.0.0.99', '10.0.0.99', '10.0.0.99'
    ],
    'dst_ip': [
        '10.0.0.1', '10.0.0.1', '10.0.0.1',
        '10.0.0.2', '10.0.0.1', '10.0.0.3',
        '10.0.0.3', '10.0.0.3', '10.0.0.3'
    ],
    'dst_port': [443, 22, 22, 80, 443, 22, 22, 22, 22],
    'bytes_sent': [1500, 3200, 3200, 450, 1200, 500, 500, 500, 500],
    'label': ['benign', 'benign', 'benign', 'benign', 'benign', 'malicious', 'malicio
}

df = pd.DataFrame(data)
df
```

Out[ ]:

	timestamp	src_ip	dst_ip	dst_port	bytes_sent	label
0	2026-07-14 10:00:01	192.168.1.10	10.0.0.1	443	1500	benign
1	2026-07-14 10:00:05	10.0.0.5	10.0.0.1	22	3200	benign
2	2026-07-14 10:00:05	10.0.0.5	10.0.0.1	22	3200	benign
3	2026-07-14 10:00:10	172.16.0.22	10.0.0.2	80	450	benign
4	2026-07-14 10:00:15	192.168.1.10	10.0.0.1	443	1200	benign
5	2026-07-14 10:00:20	10.0.0.99	10.0.0.3	22	500	malicious
6	2026-07-14 10:00:25	10.0.0.99	10.0.0.3	22	500	malicious
7	2026-07-14 10:00:30	10.0.0.99	10.0.0.3	22	500	malicious
8	2026-07-14 10:00:35	10.0.0.99	10.0.0.3	22	500	malicious

Для поиска дубликатов используется метод `duplicated()` библиотеки `pandas`:

- `subset` - названия столбцов (признаков), которые будут учитываться для выявления дубликатов (по умолчанию используются все столбцы);
- `keep` (по умолчанию `first`) - определяет, какие дубликаты (если есть) отмечать как `True`:
  - `first` - первый встречающийся экземпляр считается уникальным (`False`), все последующие дубликаты помечаются `True`;
  - `last` - последний встречающийся экземпляр считается уникальным, все предыдущие дубликаты - `True`;
  - `False` - помечать все дубликаты как `True`.

На выходе формируется логический массив, в котором повторяющиеся наблюдения обозначены как `True`.

In [ ]:
```
df.duplicated(keep = False)
```

Out[ ]:

	0
0	False
1	True
2	True
3	False
4	False
5	False
6	False
7	False
8	False

dtype: bool

Результат метода `duplicated` можно использовать как фильтр для просмотра и анализа дублирующихся наблюдений.

In [ ]:
```
df[df.duplicated(keep = False)]
```

Out[ ]:

	timestamp	src_ip	dst_ip	dst_port	bytes_sent	label
1	2026-07-14 10:00:05	10.0.0.5	10.0.0.1	22	3200	benign
2	2026-07-14 10:00:05	10.0.0.5	10.0.0.1	22	3200	benign

Строки 1 и 2 — это полные технические дубликаты. Абсолютно идентичные данные. Скорее всего, сбой агента или ретрансмиссия.

Технические дубликаты удаляются.

Для этого в `pandas` используется метод `drop_duplicates`, который возвращает `Dataframe` без дубликатов:

- `subset` - название одного или нескольких столбцов, которые учитываются для выявления и удаления дубликатов;
- `keep` - определяет, какие дубликаты (если есть) оставить:
  - `first` - удалить дубликаты, кроме первого вхождения;
  - `last` - удалить дубликаты, кроме последнего вхождения;
  - `False` - удалить все дубликаты (не остаётся ни одного экземпляра);
- `inplace` (булево, по умолчанию `False`):
  - `False` - возвращает новый `DataFrame`, исходный не меняется;
  - `True` - изменяет исходный `DataFrame` напрямую (результат не возвращается);
- `ignore_index` (по умолчанию `False`) - при удалении индексация не изменится, если `True`, то будет построен новый индекс $0, 1, \ldots, n-1$.

В нашем примере нужно удалить полные дубликаты, а так же наблюдения с одинаковыми значениями независимых признаков, на отличающимися значениями целевого признака.

In [ ]:
```
df.drop_duplicates(keep = 'first',
                   ignore_index = True,
                   inplace = True)
df
```

Out[ ]:

	timestamp	src_ip	dst_ip	dst_port	bytes_sent	label
0	2026-07-14 10:00:01	192.168.1.10	10.0.0.1	443	1500	benign
1	2026-07-14 10:00:05	10.0.0.5	10.0.0.1	22	3200	benign
2	2026-07-14 10:00:10	172.16.0.22	10.0.0.2	80	450	benign
3	2026-07-14 10:00:15	192.168.1.10	10.0.0.1	443	1200	benign
4	2026-07-14 10:00:20	10.0.0.99	10.0.0.3	22	500	malicious
5	2026-07-14 10:00:25	10.0.0.99	10.0.0.3	22	500	malicious
6	2026-07-14 10:00:30	10.0.0.99	10.0.0.3	22	500	malicious
7	2026-07-14 10:00:35	10.0.0.99	10.0.0.3	22	500	malicious

В полученной таблице строки с 4 по 7 — это семантические повторы. Один и тот же IP (10.0.0.99) "стучится" на один и тот же порт (22) с одного и того же хоста. Время между запросами — по 5 секунд. Это классический паттерн брутфорса или сканирования.

Если подать этот набор данных на вход модели «как есть», модель выучит: IP 10.0.0.99 встречается 4 раза, значит, это очень частое и нормальное поведение». Но это не нормально, это атака.

В кибербезопасности семантические дубликаты не удаляются. Наиболее распространенным подходом к их обработке является их агрегация. Т.е. осуществляется свертка повторяющихся событий во временные окна и создание новых, гораздо более информативных признаков, которые "говорят" модели «здесь много одинаковых событий подряд».

Для нашего примера сгруппируем данные по источнику,.destination и порту, и посчитаем, сколько раз это произошло за короткий промежуток времени.

In [ ]:
```
# Группируем по ключевым признакам сессии
grouped = df.groupby(
    ['src_ip', 'dst_ip', 'dst_port', 'label'],
    as_index=False
).agg(
    event_count=('timestamp', 'count'),          # Сколько раз произошло
    first_seen=('timestamp', 'min'),              # Время первого события
    last_seen=('timestamp', 'max'),               # Время последнего события
    total_bytes=('bytes_sent', 'sum')             # Суммарный объем
)

# Считаем длительность повторяющихся событий (в секундах)
grouped['duration_sec'] = (
    (grouped['last_seen'] - grouped['first_seen']).dt.total_seconds()
)

# Удаляем вспомогательные колонки с датами
grouped = grouped.drop(columns=['first_seen', 'last_seen'])
```

```
grouped
```

Out[ ]:

	src_ip	dst_ip	dst_port	label	event_count	total_bytes	duration_sec
0	10.0.0.5	10.0.0.1	22	benign	1	3200	0.0
1	10.0.0.99	10.0.0.3	22	malicious	4	2000	15.0
2	172.16.0.22	10.0.0.2	80	benign	1	450	0.0
3	192.168.1.10	10.0.0.1	443	benign	2	2700	14.0

Вместо четырех строк с брутфорсом у нас теперь одна строка.

Созданы новые признаки event_count = 4 и duration_sec = 15.

Для ML-модели это гораздо более мощные сигналы, чем просто факт четырех отдельных подключений. Алгоритм теперь явно видит: «4 попытки за 15 секунд на порт 22 — это аномалия, это похоже на брутфорс».

Таким образом, мы превратили шум в информативный признак, который модель может использовать для детекции сканирований, флуда и т.п.

Дубликаты бывают двух видов: технические (ошибки сбора) и семантические (повторяющиеся события).

Технические дубликаты необходимо удалять (drop_duplicates), чтобы не искажать веса признаков.

Семантические повторы не удаляются, а агрегируются. Мы превращаем их в новые признаки: count, duration, sum, rate. Это делает модель гораздо более устойчивой и точной.

Порядок действий: сначала обрабатываем технические дубликаты, потом агрегируем семантические.

2.2 Дубликаты в неструктурированных данных

Мы рассмотрели технологии обработки дубликатов в структурированных данных. Одрнако в реальных SIEM-системах (Splunk, ELK, MaxPatrol) львиная доля данных — это неструктурированные текстовые логи. Syslog, логи веб-серверов, кастомные логи приложений. Это просто строки текста.

При работе с неструктурированными данными различают точные, семантические и нечеткие дубликаты.

Пример возникновения точного дубликата - агент отправил одну и ту же строку дважды. В этом случае удалаяем дубли с использованием `drop_duplicates()`.

Более сложными для обработки являются семантические дубликаты.

Например, есть две записи в логах:

`Failed password for root from 192.168.1.10 port 22`

`Failed password for admin from 10.0.0.5 port 2222`

Для человека это одно и то же событие — «неудачная попытка аутентификации». Но для ML-модели это два абсолютно разных признака. Если в логах 10 000 таких строк с разными IP-адресами, модель не сможет устанвоить закономерность от «проклятия размерности» и не поймет, что это массовая атака.

Нечеткие дубликаты - это записи в логах, которые практически идентичны, но отличаются, например, лишним пробелом, версией демона (daemon) в конце строки или опечаткой.

Для обработки семантических и нечетких дубликатов используют:

- регулярные выражения;
- нормализацию и токенизацию (удаление временных меток, IP-адресов, хеширование содержимого);
- алгоритмы схожести строк (Levenshtein, Jaro-Winkler) для поиска почти одинаковых записей;
- специализированные инструменты (Drain, LogPai) для группировки похожих логов в шаблоны.

Рассмотрим технологии разбора, кластеризации и определения схожих логов.

In [ ]:
```
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Представим, что это сырые неструктурированные логи SSH и веб-сервера
raw_logs = [
    "Oct 14 10:00:01 server1 sshd[1234]: Failed password for root from 192.168.1.10 p
    "Oct 14 10:00:01 server1 sshd[1234]: Failed password for root from 192.168.1.10 p
    "Oct 14 10:00:05 server1 sshd[1235]: Failed password for admin from 10.0.0.5 port
    "Oct 14 10:00:05 server1 sshd[1236]: Failed password for admin from 10.1.1.7 port
    "Oct 14 10:00:10 server1 sshd[1237]: Failed password for root from 192.168.1.10 p
    "Oct 14 10:00:15 web1 nginx: 192.168.1.50 - - [14/Oct/2026:10:00:15] \"GET /index
    "Oct 14 10:00:20 server1 sshd[1238]: Accepted password for user1 from 172.16.0.1
]

df_logs = pd.DataFrame({'raw_log': raw_logs})
df_logs
```

Out[ ]:

	raw_log
0	Oct 14 10:00:01 server1 sshd[1234]: Failed password for root from 192.168.1.10 port 22
1	Oct 14 10:00:01 server1 sshd[1234]: Failed password for root from 192.168.1.10 port 22
2	Oct 14 10:00:05 server1 sshd[1235]: Failed password for admin from 10.0.0.5 port 2222
3	Oct 14 10:00:05 server1 sshd[1236]: Failed password for admin from 10.1.1.7 port 8080
4	Oct 14 10:00:10 server1 sshd[1237]: Failed password for root from 192.168.1.10 port 22 ssh2
5	Oct 14 10:00:15 web1 nginx: 192.168.1.50 - - [14/Oct/2026:10:00:15] "GET /index.html HTTP/1.1" 200
6	Oct 14 10:00:20 server1 sshd[1238]: Accepted password for user1 from 172.16.0.1 port 22

В этом фрагменты представлены все три типа дубликатов.

Относительно записи 0:

- запись 1 - точный дубликат;
- записи 2 и 3 - шаблонные дубликаты;
- запись 4 нечеткий дубликат (добавилось 'ssh2').

Шаг 1. Удаление точных дубликатов.

In [ ]:
```
df_logs.drop_duplicates(keep = 'first',
                        ignore_index = True,
                        inplace = True)
df_logs
```

Out[ ]:

	raw_log
0	Oct 14 10:00:01 server1 sshd[1234]: Failed password for root from 192.168.1.10 port 22
1	Oct 14 10:00:05 server1 sshd[1235]: Failed password for admin from 10.0.0.5 port 2222
2	Oct 14 10:00:05 server1 sshd[1236]: Failed password for admin from 10.1.1.7 port 8080
3	Oct 14 10:00:10 server1 sshd[1237]: Failed password for root from 192.168.1.10 port 22 ssh2
4	Oct 14 10:00:15 web1 nginx: 192.168.1.50 - - [14/Oct/2026:10:00:15] "GET /index.html HTTP/1.1" 200
5	Oct 14 10:00:20 server1 sshd[1238]: Accepted password for user1 from 172.16.0.1 port 22

Шаг 2. Извлечение шаблонов (Template Extraction).

Это ключевой этап. Мы должны заменить «переменные» (IP-адреса, порты, PID процессы, имена пользователей) на универсальные шаблоны. В промышленных системах для этого используют сложные алгоритмы, такие как Drain или Spell, но мы напишем упрощенный регулярный экспресс-парсер, чтобы показать суть подхода.

In [ ]:
```
def extract_template(log):
    # Убираем временную метку и имя хоста (они часто уникальны)
    log = re.sub(r'^[A-Za-z]{3}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2}\s+\S+\s+', '', log)

    # Заменяем IP-адреса на <IP>
    log = re.sub(r'\b(?:\d{1,3}\.){3}\d{1,3}\b', '<IP>', log)
```

```
    # Заменяем числа (PID, порты) на <NUM>
    log = re.sub(r'\b\d+\b', '<NUM>', log)
    # Заменяем пути и URL на <PATH>
    log = re.sub(r'/[\w/.-]+', '<PATH>', log)

    return log

df_logs['template'] = df_logs['raw_log'].apply(extract_template)

df_logs['template']
```

Out[ ]:

	template
0	sshd[<NUM>]: Failed password for root from <IP> port <NUM>
1	sshd[<NUM>]: Failed password for admin from <IP> port <NUM>
2	sshd[<NUM>]: Failed password for admin from <IP> port <NUM>
3	sshd[<NUM>]: Failed password for root from <IP> port <NUM> ssh2
4	nginx: <IP> - - [<NUM><PATH><NUM>:<NUM>:<NUM>:<NUM>] "GET <PATH> HTTP/<NUM>.<NUM>" <NUM>
5	sshd[<NUM>]: Accepted password for user1 from <IP> port <NUM>

dtype: object

В результате строки 0, 1 и 2 теперь превратились в один и тот же шаблон:

`sshd[<NUM>]: Failed password for <USER> from <IP> port <NUM>` .

Теперь, вместо того чтобы подавать на вход модели сырой текст, мы можем сгруппировать данные по этому шаблону и посчитать частоту однотипных событий. Для ML-модели признак «количество срабатываний шаблона Failed Password за минуту» — это мощнейший детектор брутфорса.

Шаг 3. Поиск нечетких дубликатов (Near-Duplicates).

Но что делать, если шаблонизатор не сработал? Например, лог немного изменился, и у нас появились и ssh2.

В такой ситуации необходимо использовать алгоритмы обработки естественного языка (Natural Language Processing - NLP). Например, можноиспользовать TF-IDF и косинусное сходство, чтобы найти логи, которые текстово очень похожи, даже если их шаблоны немного отличаются.

In [ ]:
```
df_logs['raw_log']
```

Out[ ]:

	raw_log
0	Oct 14 10:00:01 server1 sshd[1234]: Failed password for root from 192.168.1.10 port 22
1	Oct 14 10:00:05 server1 sshd[1235]: Failed password for admin from 10.0.0.5 port 2222
2	Oct 14 10:00:05 server1 sshd[1236]: Failed password for admin from 10.1.1.7 port 8080
3	Oct 14 10:00:10 server1 sshd[1237]: Failed password for root from 192.168.1.10 port 22 ssh2
4	Oct 14 10:00:15 web1 nginx: 192.168.1.50 - - [14/Oct/2026:10:00:15] "GET /index.html HTTP/1.1" 200
5	Oct 14 10:00:20 server1 sshd[1238]: Accepted password for user1 from 172.16.0.1 port 22

dtype: object

In [ ]:
```
# Векторизуем логи (или шаблоны, если хотим искать похожие шаблоны)
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df_logs['raw_log'])

# Считаем косинусное сходство между всеми парами логов
cos_sim = cosine_similarity(tfidf_matrix)

print("\nМатрица косинусного сходства:")
print(pd.DataFrame(cos_sim, index=df_logs.index, columns=df_logs.index).round(2))

# Находим пары с высоким сходством (например, > 0.7), но не идентичные (не 1.0)
threshold = 0.7
near_duplicates = []
for i in range(len(cos_sim)):
    for j in range(i + 1, len(cos_sim)):
        if threshold < cos_sim[i, j] < 1.0:
            near_duplicates.append((i, j, cos_sim[i, j]))

print("\nНайденные нечеткие дубликаты:")
for i, j, score in near_duplicates:
    print(f"Строка {i} и Строка {j} (Сходство: {score:.2f})")
    print(f"  -> {df_logs.iloc[i]['raw_log']}")
    print(f"  -> {df_logs.iloc[j]['raw_log']}")
```

```
Матрица косинусного сходства:
      0     1     2     3     4     5
0  1.00  0.47  0.47  0.74  0.26  0.38
1  0.47  1.00  0.70  0.49  0.18  0.33
2  0.47  0.70  1.00  0.49  0.18  0.33
3  0.74  0.49  0.49  1.00  0.28  0.38
4  0.26  0.18  0.18  0.28  1.00  0.13
5  0.38  0.33  0.33  0.38  0.13  1.00

Найденные нечеткие дубликаты:
Строка 0 и Строка 3 (Сходство: 0.74)
  -> Oct 14 10:00:01 server1 sshd[1234]: Failed password for root from 192.168.1.10 po
rt 22
  -> Oct 14 10:00:10 server1 sshd[1237]: Failed password for root from 192.168.1.10 po
rt 22 ssh2
```

Алгоритм нашел, что строка 0 и строка 3 имеют сходство 0.74. Это нечеткие дубликаты. В реальном пайплайне можно написать правило: «Если сходство > 0.7, считать эти логи одним событием и объединять их».

При работе с неструктурированным логам рекомендуется использовать следующий алгоритм:

1. Удаление полных текстовых дубликатов.
2. Создание шаблонов путем замены признаков (IP, ID, пути) на плейсхолдеры. Это снижает размерность с миллионов уникальных строк до сотен уникальных шаблонов. После чего осуществляется группировка по шаблону и подсчет частоты, скорости, уникальности источников и т.п.
3. Использование NLP для определения схожих логи, которые не удалось выявить регулярными выражениями.

3. Выявление и обработка пропущенных значений

Пропущенные значения (англ. missing values) - это отсутствующие или не зафиксированные данные в выборке.

В контексте машинного обучения они представляют проблему: большинство алгоритмов не умеют работать с пропусками в данных.

При этом нужно понимать, что в кибербезопасности пропущенное значение — это часто самый ценный сигнал атаки.

3.1 Причины возникновения пропусков

Первая группа: Технические сбои инфраструктуры.

Сетевые сканеры перегружены, в связи с чем доастаточно часто происходит потеря сетевых пакетов.

Агенты EDR на конечных точках не успели записать событие в локальный буфер перед перезагрузкой.

Канал связи с SIEM оборвался на 5 секунд.

Вторая группа: Архитектурные ограничения протоколов.

Это очень частая причина. Например, мы анализируем сетевой трафик. Но сегодня 80-90% трафика зашифровано (TLS 1.3, QUIC). Мы видим заголовки, но не видим payload (тело пакета). Для нашей модели, которая ищет сигнатуры эксплойтов в payload, эти данные будут пропущены.

Или другой пример: мы собираем логи веб-приложения, но часть API-методов не логирует User-Agent.

Третья группа: Действия злоумышленника.

Злоумышленник знает, что в компаниях собираются логи. Поэтому он чистит логи Windows Event Log после себя. Он использует Living-off-the-Land (LotL) техники, чтобы не генерировать специфичных событий. Он шифрует свой C2-трафик. В результате после

выгрузки логов для анализа вредоносной активности целые столбцы могут оказаться пустыми.

3.2 Типы пропущенных значений

Согласно классической типологии Рубена Литтла и Дональда Рубина, выделяют три типа типа пропусков.

Полностью случайные пропуски (missing completely at random, MCAR) - механизм формирования пропусков, при котором данные отсутствуют без какой-либо закономерности. При этом отсутствие данных обусловлено внешними факторами.

Ситуация.
В централизованной системе сбора логов (SIEM) обрабатываются записи о попытках аутентификации из различных корпоративных систем. Один из признаков - "client_ip" (IP-адрес клиента) в некоторых случаях оказывается пустым.

Механизм пропуска.

Пропуски в признаке "client_ip" возникают **полностью случайно** из-за временного сбоя в инфраструктуре. В момент записи лога произошёл кратковременный разрыв сетевого соединения между источником события и сервером SIEM. Из-за этого часть пакетов с логами была потеряна, а в оставшихся записях некоторые признаки (в том числе "client_ip") оказались не заполнены. Сбой носил **кратковременный и нерегулярный характер** (например, из-за перегрузки маршрутизатора в момент пиковой нагрузки).

Последствия для ML-моделей.

При MCAR **допустимо** удаление строк с пропусками без существенного смещения оценок. Однако для сохранения объёма данных предпочтительна **простая импутация** (восстановление пропусков).

Выявление пропусков типа MCAR в данных .

1. Использовать статистические тесты (например, тест Литтла) для проверки гипотезы о неслучайности пропусков.
2. Анализ закона распределения пропусков: равномерность указывает на MCAR.
3. Подтверждение технического характера сбоя (логи сетевого оборудования, отчёты об инцидентах).

MCAR в ИБ чаще всего возникает из-за **технических артефактов** (потеря пакетов, сбои сенсоров), а не из-за семантики данных. Это наименее «опасный» тип пропусков с точки зрения анализа, но требует проверки, чтобы исключить скрытые зависимости.

Случайные пропуски (missing at random, MAR) - значения признака пропущены не случайно, а ввиду некоторых закономерностей. Пропуски относят к MAR, если вероятность появления пропуска в одном признаке зависит от того, какое значение принимает другой наблюдаемый признак.

Рассмотрим пример.

Ситуация.

В системе управления инцидентами безопасности (SIEM) заполняется карточка инцидента. Один из признаков - "vulnerability_cve_id" (идентификатор уязвимости по базе данных общеизвестных уязвимостей информационной безопасности - CVE). Часть записей остаётся пустой.

Наблюдаемая закономерность.

Пропуски в признаке "vulnerability_cve_id" **систематически связаны** с другим наблюдаемым признаком - типом источника события (`source_type`):

- для событий, поступающих от **сетевых сенсоров** (IDS/IPS), поле заполняется в 90 % случаев.
- для событий от **антивирусных агентов на конечных устройствах** поле пропущено в 70 % случаев.

Механизм пропуска объясняется известным признаком - "source_type". Антивирусные агенты чаще фиксируют «поведенческие» аномалии без точного соответствия CVE, тогда как сетевые сенсоры анализируют сигнатуры, которые напрямую сопоставляются с CVE.

Зная "source_type", можно статистически оценить вероятность пропуска:

- `source_type = "IDS"` → низкая вероятность пропуска;
- `source_type = "AV_agent"` → высокая вероятность пропуска.

Последствия для ML-моделей.

Если обучать модель прогнозирования критичности без учёта механизма пропуска модель может **недооценивать риски** для событий от антивирусных агентов, т. к. отсутствие CVE будет интерпретироваться как «нет уязвимости». Корректный подход: включить признак "source_type" в модель или использовать импутацию (восстановление пропусков), учитывающую зависимость от значения "source_type".

Выявление пропусков типа MAR в данных.

1. Проверить корреляцию пропусков с **категориальными признаками** (тип источника, класс инцидента, подразделение).
2. Построить **тепловую карту пропусков** по группам (например, доля пропусков "cve_id" для каждого "source_type").
3. Применить статистические тесты (логистическая регрессия: предсказывать пропуск по наблюдаемым переменным).

Неслучайные пропуски (missing not at random, MNAR) - механизм формирования пропусков, при котором отсутствие данных напрямую зависит от самого значения признака, которое должно быть записано или скрытых факторов.

Ситуация.

Система мониторинга инцидентов (SIEM) собирает данные о кибератаках. Один из признаков - "estimated_damage" («оценочный ущерб» в денежном выражении).

Наблюдаемая закономерность.

- В 70 % случаев признак "estimated_damage" заполнен.

- В 30 % случаев - содержит пропуски.

При анализе выясняется, что пропуски **неслучайны** и коррелируют с характером инцидента. Пропуски возникают **из-за самого значения**, которое должно быть записано:

1. Аналитики сознательно оставляют поле пустым, если:

- ущерб потенциально **очень высок** , но точно оценить его на момент фиксации невозможно;
- инцидент затрагивает критически важные системы, и публичная оценка ущерба может спровоцировать репутационные риски.

2. В случаях **низкого или среднего ущерба** поле заполняется - риск ошибки невелик, и данные не чувствительны.

Последствия для ML-модели.
Если обучать модель прогнозирования ущерба на таких данных:

- Модель будет **систематически недооценивать** риск, так как самые дорогие инциденты «выпадают» из обучения.
- Возникает **смещение выборки**: данные репрезентативны только для низко- и средне-бюджетных атак.

Как выявить тип пропусков MNAR в данных.

1. Проверить, коррелируют ли пропуски с **чувствительными метриками** (ущерб, время простоя, число затронутых пользователей).
2. Проанализировать **контекст заполнения**: есть ли внутренние инструкции, допускающие пропуск значений признаков в определённых случаях.
3. Использовать статистические тесты (например, тест Литтла) для проверки гипотезы о неслучайности пропусков.

В кибербезопасности тип пропусков MNAR является самым важным, поскольку часто их наличие свидетельствует о признаках атаки.

Пример.

Злоумышленник массово запрашивает внутренние адреса веб-сервиса, чтобы проверить защиту от SSRF.

Приложение устроено так: если запрос идёт на «запрещённый» диапазон адресов, оно не выполняет внутренний HTTP-запрос, сразу возвращает ошибку клиенту и не пишет в лог поле `response_status`, потому что «запроса к бэкенду не было».Поэтому поле остаётся пустым.

Эти пропуски не случайны: они напрямую зависят от «запрещённого» URL в запросе — это типичный MNAR. Аналитик видит кластер пустых статусов для внутренних адресов и сигнализирует о разведке атаки.

3.2 Выявление пропусков

Рассмотрим механизмы выявления пропусков на примере следующего набора данных.

In [ ]:

```
rng = np.random.default_rng(42)
n = 500

df = pd.DataFrame({
    "event_time": pd.date_range(
        start="2026-07-01 08:00:00",
        periods=n,
        freq="min"
    ),

    "sensor_id": rng.choice(
        ["sensor-01", "sensor-02", "sensor-03"],
        size=n
    ),

    "protocol": rng.choice(
        ["TCP", "UDP", "ICMP"],
        size=n,
        p=[0.60, 0.25, 0.15]
    ),

    "bytes_sent": np.round(
        rng.lognormal(mean=7.4, sigma=1.2, size=n)
    ),

    "duration_sec": np.round(
        rng.exponential(scale=25, size=n),
        2
    ),

    "failed_logins": rng.poisson(
        lam=0.7,
        size=n
    ).astype(float),

    "country": rng.choice(
        ["RU", "DE", "US", "CN", "NL"],
        size=n
    ).astype(object)
})

random_attack = rng.random(n) < 0.03

df["is_attack"] = (
    (df["failed_logins"] >= 3)
    | (
        (df["bytes_sent"] > 15000)
        & (df["protocol"] == "TCP")
    )
    | random_attack
).astype(int)

# Случайная потеря данных о продолжительности соединения
mask_duration = rng.random(n) < 0.08
df.loc[mask_duration, "duration_sec"] = np.nan

# Геолокация хуже определяется для ICMP-событий
mask_country = (
    ((df["protocol"] == "ICMP") & (rng.random(n) < 0.45))
    | (rng.random(n) < 0.03)
)
df.loc[mask_country, "country"] = np.nan
```

```
# Во время некоторых атак не передается счетчик входов
mask_logins = (
    ((df["is_attack"] == 1) & (rng.random(n) < 0.30))
    | (rng.random(n) < 0.02)
)
df.loc[mask_logins, "failed_logins"] = np.nan

# Случайные сбои счетчика переданных байтов
mask_bytes = rng.random(n) < 0.05
df.loc[mask_bytes, "bytes_sent"] = np.nan
```

Признак	Содержание
`event_time`	Время события
`sensor_id`	Идентификатор сенсора
`protocol`	Сетевой протокол
`bytes_sent`	Количество переданных байтов
`duration_sec`	Продолжительность соединения
`failed_logins`	Количество неудачных входов
`country`	Страна источника

In [ ]:
```
df.head()
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
0	2026-07-01 08:00:00	sensor-01	UDP	1458.0	26.77	1.0	NaN	1
1	2026-07-01 08:01:00	sensor-03	ICMP	6334.0	41.74	0.0	DE	0
2	2026-07-01 08:02:00	sensor-02	TCP	106.0	43.22	0.0	NL	0
3	2026-07-01 08:03:00	sensor-02	TCP	272.0	34.67	0.0	RU	0
4	2026-07-01 08:04:00	sensor-02	TCP	541.0	23.65	1.0	US	0

Метод `info` показывает количество не пустых значений (`non-null`).

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   event_time     500 non-null    datetime64[ns]
 1   sensor_id      500 non-null    object
 2   protocol       500 non-null    object
 3   bytes_sent     470 non-null    float64
 4   duration_sec   461 non-null    float64
 5   failed_logins  475 non-null    float64
 6   country        449 non-null    object
 7   is_attack      500 non-null    int64
dtypes: datetime64[ns](1), float64(3), int64(1), object(3)
memory usage: 31.4+ KB
```

Для выявления пропусков используется метод `isna()`, который возвращает булеву метку: `False` - нет пропуска, `True` - есть пропуск.

Этот метод распознает как пустые ячейки, так и типы `NaN` как пропущенные значения.

In [ ]:
```
df.isna()
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_atta
0	False	False	False	False	False	False	True	Fal
1	False	False	False	False	False	False	False	Fal
2	False	False	False	False	False	False	False	Fal
3	False	False	False	False	False	False	False	Fal
4	False	False	False	False	False	False	False	Fal
...	...	...	...	...	...	...	...	
495	False	False	False	False	False	False	False	Fal
496	False	False	False	False	False	False	False	Fal
497	False	False	False	False	True	True	False	Fal
498	False	False	False	False	False	False	False	Fal
499	False	False	False	False	True	False	False	Fal

500 rows × 8 columns

Такой формат не позволяет провести анализ пропусков, поэтому переходят к числовым характеристикам.

Для каждого признака желательно определить не только количество, но и долю пропущенных значений.

In [ ]:
```
missing_report = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_percent": df.isna().mean() * 100
})

missing_report = missing_report.sort_values(
```

```
    "missing_percent",
    ascending=False
)

missing_report.round(2)
```

Out[ ]:

	missing_count	missing_percent
country	51	10.2
duration_sec	39	7.8
bytes_sent	30	6.0
failed_logins	25	5.0
protocol	0	0.0
sensor_id	0	0.0
event_time	0	0.0
is_attack	0	0.0

Так же можно отобразить количество пропусков в каждой строке.

In [ ]:
```
df.isnull().sum(axis = 1).head(7)
```

Out[ ]:

	0
0	1
1	0
2	0
3	0
4	0
5	0
6	1

dtype: int64

Скрытые обозначения пропусков

Не все пропуски автоматически распознаются как `NaN`. В реальных журналах могут использоваться:

- пустая строка;
- дефис;
- строка N/A;
- значение unknown;
- специальное число, например -1 или 999999.

Перед обработкой такие обозначения нужно привести к единому виду.

In [ ]:
```
df = df.replace({
    "": np.nan,
    "-": np.nan,
    "N/A": np.nan,
    "null": np.nan
})
```

3.3 Определение типа пропусков

Для определенеия типа пропусков используются различные подходы:

- визуальная экспресс-оценка;
- количественная оценка: корреляция индикаторов пропусков с наблюдаемыми признаками.

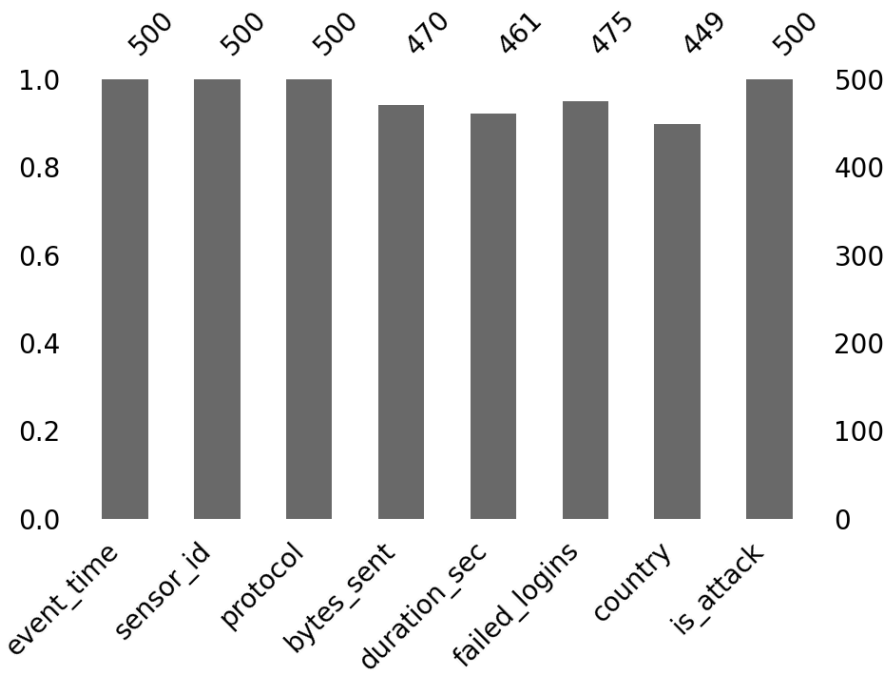
Визуальная экспресс-оценка.

Для визуалиции распределения пропусков на практике чаще всего используется специальная бибилиотека Missingno.

Missingno - библиотека `Python`, которая помогает визуализировать недостающие данные в датафреймах `pandas` и определить частоту и расположение пропущенных значений в данных.

In [ ]:
```
import missingno as msno
msno.bar(df, figsize = (5, 3), fontsize = 10)
```

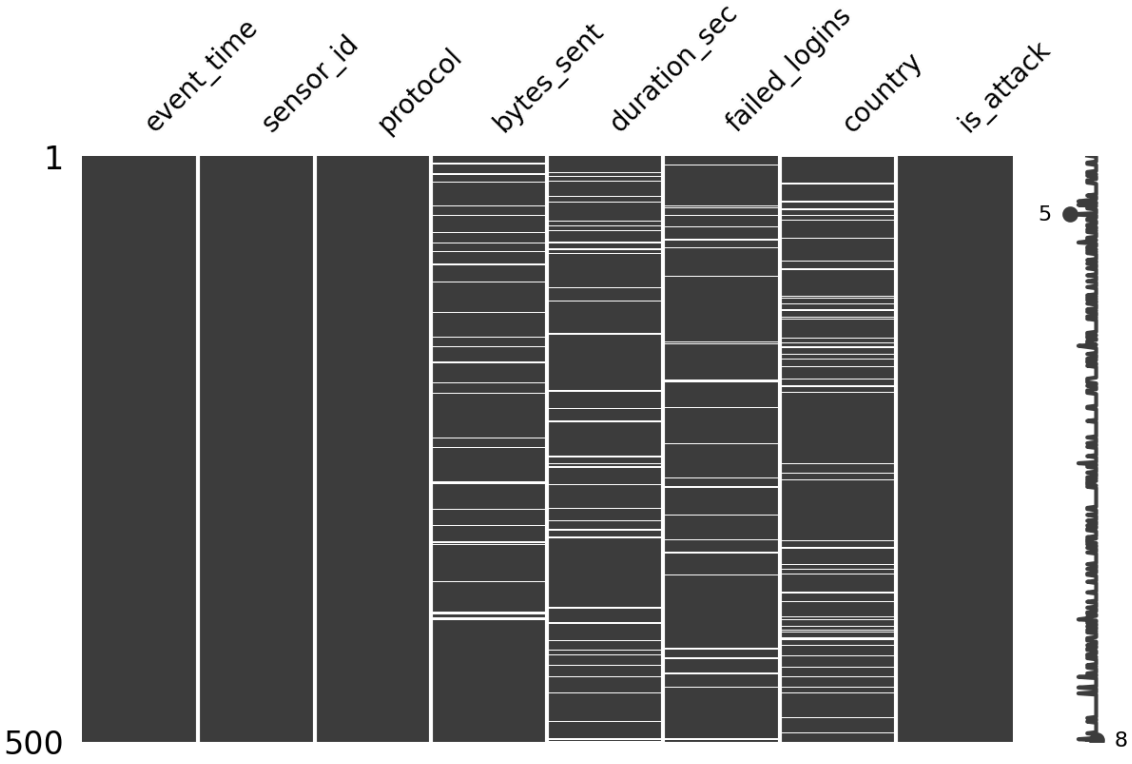
Out[ ]:
```
<Axes: >
```



Метод `matrix()` позволяет оценить случайность возникновения пропущенных значений.

In [ ]:
```
msno.matrix(df, figsize = (7, 4), fontsize = 10)
```

Out[ ]:
```
<Axes: >
```



1. Тёмные блоки: означают, что в этих строках и столбцах данные есть (нет пропусков).
2. Тонкие белые полоски: это пропуски (`NaN`) — именно их и визуализирует матрица.
3. Шкала справа показывает процент непустых ячеек в строке. Например, отметка «5» соответствует 5 % заполненности, «8» — 8 %.

Видно, что для всех признаков пропуски распределены не случайно, а «полосами».

1. Признак "bytes_sent": вероятен тип MAR (предположительно зависит от protocol) или MNAR (если пропуск означает «не применимо» и это скрыто в логике сенсора).
2. Признак "duration_sec": аналогично "bytes_sent" пропуски скорее всего зависят от протокола или намеренного отсутствия тайминга, т.е. тип - MAR/MNAR.
3. Признак "failed_logins": если пропуски характерны для конкретных сенсоров или типов событий, то тип MAR; если это намеренное отсутствие логирования при атаках - MNAR.
4. Признак "country": если пропуски связаны с отсутствием GeoIP-данных для некоторых сенсоров, то тип MAR; если это целенаправленное сокрытие геолокации,то - MNAR.

Количественная оценка.

Для каждого столбца с пропусками создаём бинарный индикатор `missing` , который равен 1, если пропуск и 0 - если нет пропуска.

In [ ]:
```
df_missing_flag = df.copy()
for col in ['bytes_sent', 'duration_sec', 'failed_logins', 'country']:
    df_missing_flag[f'{col}_missing'] = df_missing_flag[col].isna()
```

In [ ]:
```
df_missing_flag.head()
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
0	2026-07-01 08:00:00	sensor-01	UDP	1458.0	26.77	1.0	NaN	1
1	2026-07-01 08:01:00	sensor-03	ICMP	6334.0	41.74	0.0	DE	0
2	2026-07-01 08:02:00	sensor-02	TCP	106.0	43.22	0.0	NL	0
3	2026-07-01 08:03:00	sensor-02	TCP	272.0	34.67	0.0	RU	0
4	2026-07-01 08:04:00	sensor-02	TCP	541.0	23.65	1.0	US	0

Для индикаторных столбцов числовых признаков вычисляем коэффициенты корреляция Пирсона, а для категориальных — доли пропусков по группам признаков "protocol" и "sensor_id".

In [ ]:
```
for col in ['bytes_sent', 'duration_sec', 'failed_logins', 'country']:
    # Для числовых признаков — корреляция Пирсона
    if df_missing_flag[col].dtype in ['float64', 'int64']:
        # Select only numeric and boolean columns for correlation to avoid ValueError
        numeric_and_bool_df = df_missing_flag.select_dtypes(include=['number', 'bool'
        corr = numeric_and_bool_df.corr()[f'{col}_missing'].drop(f'{col}_missing')
        print(f"Корреляции пропусков {col}:\n{corr.sort_values(ascending=False).head(
    # Для категориальных — доля пропусков по группам
    else:
        print(f"Доля пропусков {col} по protocol:\n{df_missing_flag.groupby('protocol
        print(f"Доля пропусков {col} по sensor_id:\n{df_missing_flag.groupby('sensor_
```

```
Корреляции пропусков bytes_sent:
failed_logins_missing    0.019320
country_missing         -0.001670
is_attack               -0.014120
duration_sec_missing    -0.042081
duration_sec            -0.044042
failed_logins           -0.056934
bytes_sent                    NaN
Name: bytes_sent_missing, dtype: float64
Корреляции пропусков duration_sec:
country_missing          0.000542
bytes_sent              -0.005722
failed_logins           -0.029605
failed_logins_missing   -0.032508
is_attack               -0.032565
bytes_sent_missing      -0.042081
duration_sec                  NaN
Name: duration_sec_missing, dtype: float64
Корреляции пропусков failed_logins:
is_attack               0.466583
bytes_sent              0.046941
bytes_sent_missing      0.019320
duration_sec           -0.000708
country_missing        -0.016677
duration_sec_missing   -0.032508
failed_logins                NaN
Name: failed_logins_missing, dtype: float64
Доля пропусков country по protocol:
protocol
ICMP    0.486486
TCP     0.040268
UDP     0.023438
Name: country_missing, dtype: float64
Доля пропусков country по sensor_id:
sensor_id
sensor-01    0.112426
sensor-02    0.103659
sensor-03    0.089820
Name: country_missing, dtype: float64
```

Анализ корреляций индикаторов пропусков.

Признак "bytes_sent_missing": все корреляции близки к 0.

Вывод: нет выраженной линейной связи с наблюдаемыми признаками. Это не сильный аргумент в пользу MAR, но и не доказывает MCAR (в кибербезопасности MCAR почти не бывает). Вероятнее MNAR: пропуски зависят от скрытого фактора.

Признак "duration_sec_missing": все корреляции близки к 0.

Вывод: аналогично "bytes_sent" — нет явной связи с наблюдаемыми признаками. Скорее всего MNAR.

Признак "failed_logins_missing": здесь есть важный момент — корреляция с "is_attack" равна 0.4666. Это сильная положительная связь: чем выше вероятность атаки ("is_attack"), тем чаще отсутствует значение "failed_logins".

Вывод: это классический MAR: пропуск зависит от наблюдаемого признака "is_attack". Возможно, при атаках логирование неудачных входов отключается, либо эти события попадают в другой поток логов.

Анализ долей пропусков по группам.

Смотрим на доли пропусков по признакам "protocol" и "sensor_id" для признака "country":

country_missing по protocol:

- ICMP: 48.6 % пропусков;
- TCP: 4.0 %;
- UDP: 2.3 %.

Вывод: пропуски зависят от наблюдаемого признака "protocol" — это явный признак MAR. Для ICMP часто нет полноценной GeoIP-информации (или она не собирается).

country_missing по sensor_id:

- sensor-01: 11.2 %;
- sensor-02: 10.4 %;
- sensor-03: 9.0 %.

Вывод: различия между сенсорами есть, но они не такие значительные, как по протоколам. Это дополнительный аргумент в пользу MAR (пропуск зависит от сенсора), но основной драйвер — протокол.

Что можно сделать с пропусками?

1. Игнорировать.
2. Удалить.
3. Чем-то заполнить.

3.4 Игнорирование пропусков

Важно учитывать, что многие методы `Python` игнорируют пропуски.

Например, используем методы `groupby` и `count` для того, чтобы посчитать количество различных типов протоколов в наборе данных.

In [ ]:
```
df.groupby('protocol').count()
```

Out[ ]:

protocol	event_time	sensor_id	bytes_sent	duration_sec	failed_logins	country	is_attack
ICMP	74	74	69	70	71	38	74
TCP	298	298	285	270	281	286	298
UDP	128	128	116	121	123	125	128

Таким образом, метод `count` игнорировал пропуски и не выдал ошибку. По такому же принципу работают все методы библиотеки `pandas`, вычисляющие числовые характеристики.

Игнорирование пропусков

- Плюсы: не надо думать об алгоритмах импутации (восстановления пропусков) - данные сразу подаются на вход алгоритма машинного обучения. Однако, большинство моделей не работают с данными, имеющими пропуски.
- Минусы: нужно понимать, как именно применяемый метод или модель машинного обучения обработают недостающие данные. В противном случае есть риск получить какие-нибудь артефакты и даже не узнать, какие именно.

3.5 Удаление пропусков

Для удаления пропусков используется метод `dropna()`:

- `axis` (по умолчанию 0) - ось таблицы:
  - 0 или `index` - удалять строки с пропусками;
  - 1 или `columns` - удалять столбцы с пропусками;
- `how` (по умолчанию `any`) - стратегия удаления пропусков:
  - `any` - удалить строку/столбец, если есть хотя бы одно пропущенное значение;
  - `all` - удалить строку/столбец, только если пропущены все значения;
- `thresh` (по умолчанию `None`) - минимальное число непропущенных значений в строке/столбце для сохранения. Если в строке/столбце меньше `thresh` непропущенных значений, то происходит их удаление;
- `subset` (по умолчанию `None`) - список столбцов (или индексов), в которых нужно удалить пропуски. Удаление происходит только если пропуск найден в указанных столбцах;
- `inplace` (по умолчанию `False`):
  - `False` - возвращается новый объект `DataFrame`, исходный не меняется;
  - `True` - изменяет исходный объект `DataFrame` и возвращает `None`.

Удаление объектов (строк)

По умолчанию удаляются строки (параметр `axis = 0`), в которых пропущено хотя бы одно значение.

In [ ]:
```
df_dropped = df.dropna()
print(f"Осталось строк: {len(df_dropped)} из {len(df)}")
```

```
Осталось строк: 367 из 500
```

Недостатком удаления строк с пропущенными значениями является возможное сильное сокращение количества наблюдений.

В кибербезопасности, где аномалии и так редки (проблема несбалансированных классов), удалять строки не рекомендуется. В противном случае мы удалим часть атакующего трафика из обучения.

Использование параметра `how = 'all'` позволяет удалить только те строки, в которых пропущены все значения.

In [ ]:
```
df_dropped = df.dropna(how = 'all')
print(f"Осталось строк: {len(df_dropped)} из {len(df)}")
```

```
Осталось строк: 500 из 500
```

Параметр `subset` позволяет осуществлять удаление строк, в которых пропущены значения в конкретных столбцах.

In [ ]:
```
df_dropped = df.dropna(subset = ['duration_sec'])
print(f"Осталось строк: {len(df_dropped)} из {len(df)}")
```

```
Осталось строк: 461 из 500
```

Удаление признаков (столбцов)

Для удаления столбцов так же используется метод `dropna()`, но с параметром `axis = 1`.

In [ ]:
```
df_dropped = df.dropna(axis = 1)
print(f"Осталось строк: {len(df_dropped)} из {len(df)}")
print(f"Осталось столбцов: {len(df_dropped.columns)} из {len(df.columns)}")
```

```
Осталось строк: 500 из 500
Осталось столбцов: 4 из 8
```

Использование параметра `how = 'all'` позволяет удалить только те признаки, в которых пропущены все значения.

In [ ]:
```
df_dropped = df.dropna(axis = 1, how = 'all')
print(f"Осталось строк: {len(df_dropped)} из {len(df)}")
print(f"Осталось столбцов: {len(df_dropped.columns)} из {len(df.columns)}")
```

```
Осталось строк: 500 из 500
Осталось столбцов: 8 из 8
```

На практике удаление признаков осуществляется, как правило, при условии превышения количества пропусков некоторого заданного порога.

Например, проверим, есть ли признаки, в которых более 75 % пропусков и удалим их.

Для этого можно использовать параметр `thresh`.

Например, `thresh = len(dft) // 4` означает, что будут удалены столбцы, в которых примерно менее 25% заполненных значений (75 и более % пропусков).

In [ ]:
```
df_dropped = df.dropna(thresh = len(df) // 4, axis = 1)
print(f"Осталось строк: {len(df_dropped)} из {len(df)}")
print(f"Осталось столбцов: {len(df_dropped.columns)} из {len(df.columns)}")
```

```
Осталось строк: 500 из 500
Осталось столбцов: 8 из 8
```

3.6 Заполнение пропусков

Как было сказано выше, удаление пропусков в кибербезопасности делать не рекомендуется. В этом случае прибегают к восстановлению пропущенных значений (missing values imputation).

Одномерные методы

Одномерные методы (Single Imputation) — это заполнение пропусков в одном признаке только на основе статистики самого этого признака, без учёта других столбцов.

Формально для признака с пропусками мы вычисляем некоторую статистику по наблюдаемым значениям и подставляем её вместо `NaN`.

Основные одномерные методы:

- заполнение константой (часто 0 или специальным маркером вроде −1, "unknown");
- заполнение средним (mean) — для числовых признаков с симметричным распределением;
- заполнение медианой (median) — устойчиво к выбросам, часто лучше среднего;
- заполнение модой (mode) — для категориальных признаков;
- заполнение случайным значением из наблюдаемого распределения (менее распространено).

Для замены пропущенных значений (числовых и строковых) используется метод `fillna`:

- `value` (`None`) - значение, используемое для заполнения пропусков;
- `method` (`None`) - способ заполнения пропусков:
  - `ffill` - заполнить пропуск последним допустимым значением признака;
  - `bfill` - заполнить пропуск следующим допустимым значением признака;
- `axis` (`None`) - ось, вдоль которой заполняются пропущенные значения (для серий этот параметр не используется и по умолчанию равен 0);
- `inplace` (`False`) - позволяет осуществлять перезапись пропусков указанными значениями на месте при установке `inplace = True`.

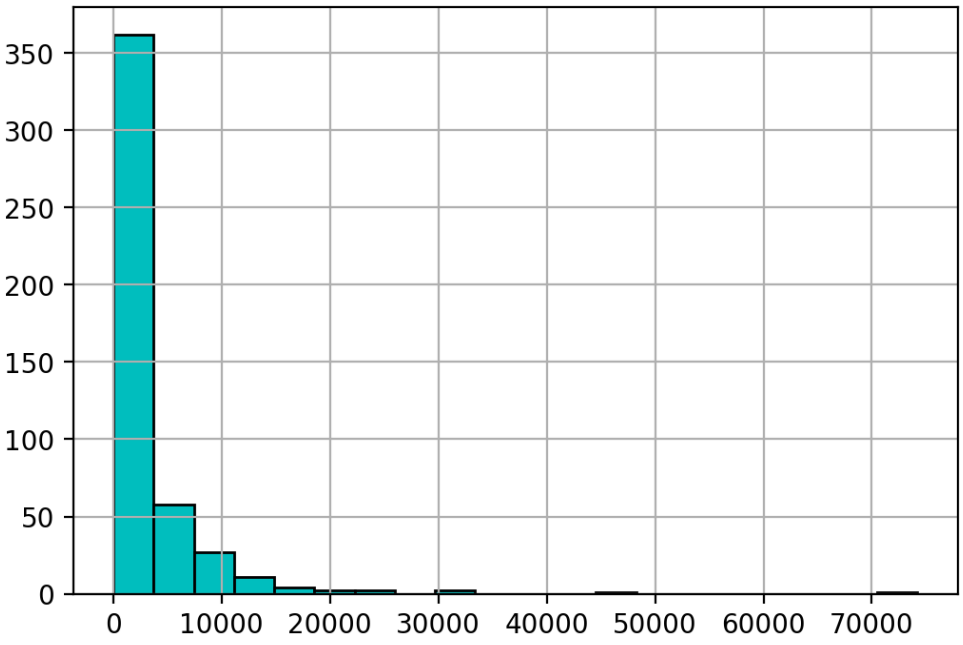
Выбор типа одномерного метода зависит от природы пропусков. Для пропусков типа MCAR можно использовать средние значения. Для пропусков типа MAR/MNAR, как правило, пропуски заменяются константой.

Заполнение средним и константой

Рассмотрим технологию заполнения пропусков для признака количественных признаков на примере признака "bytes_sent".

Если есть уверенность, что пропуски относятся к типу MCAR, то нужно проанализировать закон распределения признака и выбрать либо среднее значение, либо медиану.

In [ ]:
```
plt.figure(figsize = (6, 4))
plt.hist(df['bytes_sent'], bins = 20,
         color = 'c',
         edgecolor = 'black')
plt.grid()
```



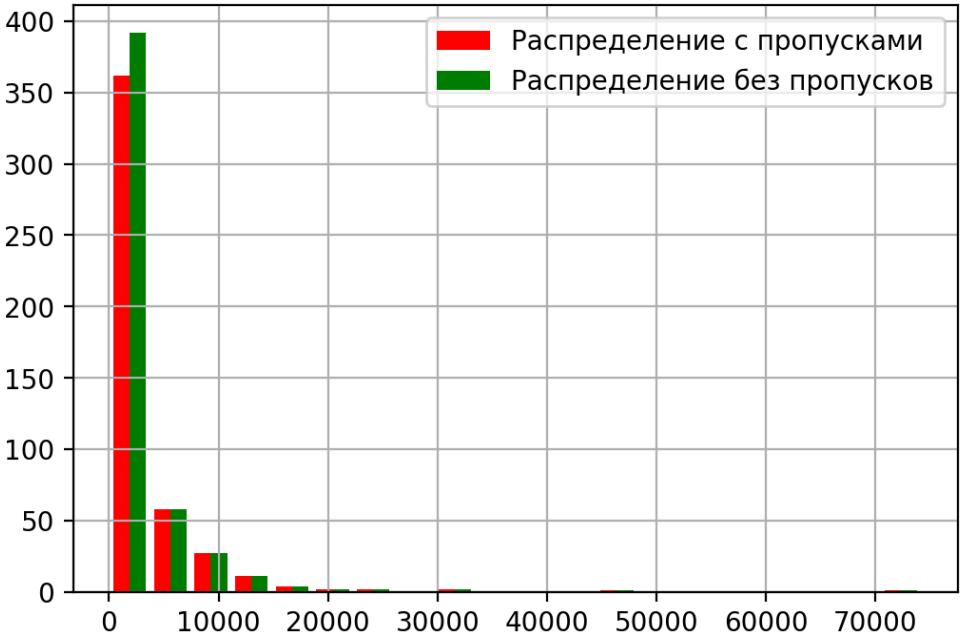
Так как распределение значительно отличается от нормального, то целесообразно заменить пропуски медианой.

In [ ]:
```
df_median = df.copy()
df_median['bytes_sent'] = df['bytes_sent'].fillna(df['bytes_sent'].median())
df_median['bytes_sent'].isna().sum()
```

Out[ ]:
```
np.int64(0)
```

Недостатком замены пропусков средними значениями является изменение исходного распределения значений признака.

In [ ]:
```
plt.figure(figsize = (6, 4))
plt.hist([df['bytes_sent'], df_median['bytes_sent']],
          bins = 20, color = ['r', 'g'])
plt.legend(['Распределение с пропусками',
            'Распределение без пропусков'])
plt.grid()
```



Если есть предположение, что пропуски относятся к типу MNAR, то для замены пропусков необходимо использовать константу.

Например, можно использовать 0 если пропуск означает «ничего не отправлено» (например, ICMP-пакет без полезной нагрузки).

In [ ]:
```
df_const = df.copy()
df_const['bytes_sent'] = df['bytes_sent'].fillna(0)
df_const['bytes_sent'].isna().sum()
```

Out[ ]:
```
np.int64(0)
```

Рассмотрим технологию заполнения пропусков для признака категориальных признаков на примере признака "country".

Для категориальных признаков заполнение пропусков осуществляется либо модой, либо константой.

В подавляющем большинстве случае рекомендуется использовать замену константой.

Для признака "country" (MAR по "protocol") замена пропусков модой может скрыть важные паттерны, например, атаки из редких стран.

Поэтому правильнее заменить константой типа 'unknown', чтобы явно обозначить отсутствие GeoIP-данных.

In [ ]:
```
df_const_cat = df.copy()
df_const_cat['country'] = df['country'].fillna('unknown')
df_const_cat['country'].isna().sum()
```

Out[ ]:
```
np.int64(0)
```

Важные предостережения для задач кибербезопасности.

1. Не удаляйте строки с пропусками. В детекции атак такие строки часто самые информативные (например, атаки с сокрытием признаков).

2. Всегда создавайте флаги пропусков. Даже если вы заполняете пропуски константой, флаг сам по себе может быть сильным предиктором.
3. Учитывайте тип пропусков. Для MNAR (как bytes_sent, duration_sec) заполнение средним/медианой может быть опасным — лучше использовать константы и флаги.

Заполнение групповыми числовыми характеристиками

Суть в том, чтобы разбить данные на группы по одному или нескольким признакам, а затем заполнять пропуски в каждой группе отдельно — используя статистику только внутри этой группы (среднее, медиану, моду). Идея в том, что внутри однородной группы значения признака ведут себя схоже, и локальная статистика лучше отражает суть, чем общая по всей выборке. Это особенно уместно, если механизм пропусков связан с группирующим признаком (т.е. имеет смысл MAR).

Для реализации этого подхода чаще всего используют комбинацию `groupby` и `transform`. Метод `transform` позволяет применить функцию к каждой группе и вернуть результат той же длины, что и исходный `DataFrame`, что идеально для замены значений на месте.

Пример заполнения пропусков в признаке "bytes_sent".

Можно сгруппировать значения признака по "sensor_id" или "protocol". Это объясняется тем, что у разных сенсоров или для разных протоколов типичный объём передаваемых данных может сильно отличаться. Заполнение медианой внутри такой группы учтёт эту специфику лучше, чем глобальная медиана.

In [ ]:
```
df_median_gr = df.copy()
df_median_gr['bytes_sent'] = df.groupby(['sensor_id', 'protocol'])['bytes_sent'].tran
    lambda x: x.fillna(x.median()))

df_median_gr['bytes_sent'].isna().sum()
```

Out[ ]:
```
np.int64(0)
```

Пример заполнения пропусков в признаке "country".

Для этого признака, исходя из результатов анализа его распределения, группировку необходимо осуществлять по признаку "protocol".

Такой выбор объясняется технической логикой. Для ICMP-пакетов часто не делают полноценный GeoIP-lookup (или он не применим в том же формате), тогда как для TCP-сессий определение страны — стандартная операция. Пропуск здесь не случайность, а следствие типа трафика.

Так же статистика показывает разницу почти в 20 раз между ICMP и остальными протоколами. Глобальная мода (самая частая страна во всём датасете) будет почти наверняка «перетягивать» данные для ICMP, искажая их. Групповая мода даст более корректный результат.

Создадим функцию, которая будет возвращать моду по всем значениям признака (возвращает первое значение, если их несколько). Если мод нет (например, все значения уникальны или столбец полностью пустой), возвращается строка 'unknown'.

In [ ]:
```
def get_mode(x):
    return x.mode()[0] if not x.mode().empty else 'unknown'
```

Замена пропусков групповыми модами.

In [ ]:
```
df_moda_gr = df.copy()
df_moda_gr['country'] = df.groupby('protocol')['country'].transform(
    lambda x: x.fillna(get_mode(x))
)

df_moda_gr['country'].isna().sum()
```

Out[ ]:
```
np.int64(0)
```

Индикаторный метод

Индикаторный метод (англ. indicator method, missingness indicator) - это подход к обработке пропусков, который заключается в следующем:

- для каждого столбца с пропусками создаётся дополнительный столбец, содержащий бинарный индикатор (1 - в исходном признаке пропуск был, 0 - значение есть);
- исходные пропуски заполняются некоторым замещающим значением.

Рассмотрим пример использования индикаторного метода для обработки пропусков в признаке "country".

Создаём индикаторную переменную "country_missing": 1 — было пропущено, 0 — значение было.

In [ ]:
```
# isna() даёт булев Series, преобразуем в 0/1
df_missing = df.copy()
df_missing['country_missing'] = df['country'].isna().astype(int)
df_missing.head()
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
0	2026-07-01 08:00:00	sensor-01	UDP	1458.0	26.77	1.0	NaN	1
1	2026-07-01 08:01:00	sensor-03	ICMP	6334.0	41.74	0.0	DE	0
2	2026-07-01 08:02:00	sensor-02	TCP	106.0	43.22	0.0	NL	0
3	2026-07-01 08:03:00	sensor-02	TCP	272.0	34.67	0.0	RU	0
4	2026-07-01 08:04:00	sensor-02	TCP	541.0	23.65	1.0	US	0

Заполняем пропуски в исходной колонке, например, модой внутри группы по "protocol".

In [ ]:
```
df_missing['country'] = df_missing.groupby('protocol')['country']. \
transform(lambda x: x.fillna(x.mode() if not x.mode().empty else 'unknown'))

df_missing.head()
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
0	2026-07-01 08:00:00	sensor-01	UDP	1458.0	26.77	1.0	RU	1
1	2026-07-01 08:01:00	sensor-03	ICMP	6334.0	41.74	0.0	DE	0
2	2026-07-01 08:02:00	sensor-02	TCP	106.0	43.22	0.0	NL	0
3	2026-07-01 08:03:00	sensor-02	TCP	272.0	34.67	0.0	RU	0
4	2026-07-01 08:04:00	sensor-02	TCP	541.0	23.65	1.0	US	0

В модель машинного обучения подаются и заполненные исходные столбцы, и новые индикаторные.

Цель - дать модели явный сигнал о том, где были пропуски, чтобы она могла учесть этот паттерн.

Индикаторный метод рекомендуется применять для пропусков типа MAR или MNAR: их наличие несёт информацию. Этот метод не стоит использовать, когда пропуски строго случайны (MCAR) и их мало (< 5 %).

Заполнение пропусков средствами библиотеки sklearn

Библиотека `sklearn`, которая является основной для построения моделей машинного обучения, имеет в своем составе средства работы с пропусками.

Рассмотренные выше одномерные методы заполнения пропусков реализованы в классе `SimpleImputer`:

- `missing_values` (по умолчанию `np.nan`) - определяет, каким образом обозначаются пропуски (можно задать любое обозначение, как строковое, так и числовое);
- `strategy` (по умолчанию `mean`) - определяет статистику, которая будет использоваться для заполнения пропусков:
  - `mean` - среднее арифметическое;
  - `median` - медиана;
  - `most_frequent` - наиболее часто встречающееся значение (если таких значений несколько, то возвращается только наименьшее);
  - `constant` - замена пропусков на `fill_value`;
- `fill_value` - константное значение, которое заменит пропущенные значения (по умолчанию `None`, что соответствует значению 0);

- `add_indicator` (по умолчанию `False`) - если значение `True`, то к результату добавляется бинарный индикатор пропусков (дополнительный столбец для каждого признака с пропусками). Полезно для моделей, которые могут использовать информацию о наличии пропусков. При вызове метода `transform()` возвращается массив, где последние столбцы - индикаторы (1 — пропуск был, 0 — не было).

Рассмотрим пример заполнения пропусков в признаке "bytes_sent".

In [ ]:
```
df.head(7)
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
0	2026-07-01 08:00:00	sensor-01	UDP	1458.0	26.77	1.0	NaN	1
1	2026-07-01 08:01:00	sensor-03	ICMP	6334.0	41.74	0.0	DE	0
2	2026-07-01 08:02:00	sensor-02	TCP	106.0	43.22	0.0	NL	0
3	2026-07-01 08:03:00	sensor-02	TCP	272.0	34.67	0.0	RU	0
4	2026-07-01 08:04:00	sensor-02	TCP	541.0	23.65	1.0	US	0
5	2026-07-01 08:05:00	sensor-03	UDP	9447.0	52.33	0.0	US	0
6	2026-07-01 08:06:00	sensor-01	UDP	NaN	25.00	0.0	NL	0

Как было показано выше, распределение признака не является нормальным, поэтому зададим стретегию заполнения пропусков медианой.

In [ ]:
```
from sklearn.impute import SimpleImputer
si = SimpleImputer(missing_values = np.nan,
                   strategy = 'median')
si
```

Out[ ]:
```
SimpleImputer
SimpleImputer(strategy='median')
```

Метод `fit()` вычисляет выбранные числовые характеристики признака (в нашем примере медиану).

In [ ]:
```
df_imputer = df.copy()
si.fit(df_imputer[['bytes_sent']])

si.statistics_ # значение вычисленной статистики
```

Out[ ]:
```
array([1650.5])
```

Заполнение пропусков осуществляется с помощью метода `transform()`.

In [ ]:
```
df_imputer['bytes_sent'] = si.transform(df_imputer[['bytes_sent']])
df_imputer.iloc[[6]]
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
6	2026-07-01 08:06:00	sensor-01	UDP	1650.5	25.0	0.0	NL	0

In [ ]:
```
df_imputer['bytes_sent'].isna().sum()
```

Out[ ]:
```
np.int64(0)
```

Класс `SimpleImputer` позволяет реализовать индикаторный метод. Для этого необходимо указать параметр `add_indicator = True` - добавить индикатор.

Применим индикаторный метод для признака "bytes_sent" и произведем замену пропусков медианой.

Можно использовать метод `fit_transform()`, который объединяет два метода. На выходе получаем дмумерный массив, который содержит заполненные медианой пропуски и дополнительный столбец с индикатором: 0 - не было пропуска, 1 - был пропуск.

In [ ]:
```
df_imputer_ind = df.copy()
si = SimpleImputer(strategy = 'median',
                   add_indicator = True)
si.fit_transform(df_imputer_ind[['bytes_sent']])[:7]
```

Out[ ]:
```
array([[1.4580e+03, 0.0000e+00],
       [6.3340e+03, 0.0000e+00],
       [1.0600e+02, 0.0000e+00],
       [2.7200e+02, 0.0000e+00],
       [5.4100e+02, 0.0000e+00],
       [9.4470e+03, 0.0000e+00],
       [1.6505e+03, 1.0000e+00]])
```

Для удобства результат можно представить в виде таблицы.

In [ ]:
```
df_imputer_ind[['bytes_sent','bytes_sent_miss']] = si.fit_transform(df_imputer_ind[['
df_imputer_ind.head(7)
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
0	2026-07-01 08:00:00	sensor-01	UDP	1458.0	26.77	1.0	NaN	1
1	2026-07-01 08:01:00	sensor-03	ICMP	6334.0	41.74	0.0	DE	0
2	2026-07-01 08:02:00	sensor-02	TCP	106.0	43.22	0.0	NL	0
3	2026-07-01 08:03:00	sensor-02	TCP	272.0	34.67	0.0	RU	0
4	2026-07-01 08:04:00	sensor-02	TCP	541.0	23.65	1.0	US	0
5	2026-07-01 08:05:00	sensor-03	UDP	9447.0	52.33	0.0	US	0
6	2026-07-01 08:06:00	sensor-01	UDP	1650.5	25.00	0.0	NL	0

Многомерные методы

Многомерные методы (Multivariate Imputation) основаны на заполнении пропусков одного признака на основе данных других признаков.

Многомерные методы позволяют выявлять закономерности: например, если для ряда записей с `protocol='ssh'` и `bytes_sent=1024` значение `duration_sec` всегда было около 5 секунд, то при пропуске `duration_sec` в похожей строке алгоритм предложит близкое значение. Так сохраняется структура данных лучше, чем при одномерных методах.

Наиболее распространенные многомерные методы восстановления пропусков:

- KNN (Nearest Neighbors) - использование метода k-ближайших соседей;
- MICE (Multiple Imputation by Chained Equations - множественная импутаия с помощью цепных уравнений) - использование модели линейной регрессии;
- MissForest - использование модели Random Forest - случайный лес;
- GAN (Generative Adversarial Networks) - использование генеративно-состязательных сетей для моделирования распределения данных и генерации реалистичных значений пропусков.

Рассмотрим технологию восстановления пропусков на примере метода MICE.

Алгоритм метода MICE.

1. Инициализация. Все пропуски во всех признаках временно заполняются простыми методами (например, средним, медианой или модой для категориальных данных). Это нужно, чтобы получить полностью заполненную матрицу для первого запуска модели.

2. Циклическая импутация. Алгоритм последовательно обрабатывает каждый признак с пропусками:

- выбирается признак с пропущенными значениями и значения-заполнители изменяются на пропуски (`NaN`);
- на полных строках обучается регрессионная модель, в которой в качестве зависимой переменной выступает текущий признак, а остальные признаки являются предикторами;
- модель предсказывает пропуски в текущем признаке;
- пропуски заменяются полученными предсказаниями.

3. Итерация. Процедуры шага 2 повторяются для других признаков с пропусками. После обработки всех признаков один цикл завершается.

4. Повтор циклов. Описанные циклы повторяются несколько раз, каждый раз используя новые заполненные значения в качестве отправной точки. Это происходит до тех пор, пока разницы между двумя последними пропущенными значениями не будут близки к нулю. Как правило, после 10 циклов модель достигает сходимости, и оценки больше не улучшаются.

Дополнительно рекомендуется повторить алгоритм несколько раз. При этом каждый раз пропуски инициализируются случайными значениями из апостериорного распределения. В результате получается нексколько импутированных датасетов (чтобы получить вариативность). Конечные значения пропусков можно определить простым усреднением.

Метод MICE реализован в классе `IterativeImputer` библиотеки `sklearn` .

Этот класс является экспериментальным и для его использования нужно явно импортировать `enable_iterative_imputer` .

In [ ]:
```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
```

Основные параметры класса `IterativeImputer` :

- `estimator` (по умолчанию линейная регрессия - `LinearRegression`) - модель для предсказания пропусков;
- `initial_strategy` (по умолчанию `mean`) - метод начального заполнения пропусков;
- max_iter (по умолчанию 10) - число итераций до остановки.

В качестве модели для предсказания пропусков может использоваться любая модель машинного обучения, реализованная в библиотеке `sklearn` .

In [ ]:
```
df.head()
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
0	2026-07-01 08:00:00	sensor-01	UDP	1458.0	26.77	1.0	NaN	1
1	2026-07-01 08:01:00	sensor-03	ICMP	6334.0	41.74	0.0	DE	0
2	2026-07-01 08:02:00	sensor-02	TCP	106.0	43.22	0.0	NL	0
3	2026-07-01 08:03:00	sensor-02	TCP	272.0	34.67	0.0	RU	0
4	2026-07-01 08:04:00	sensor-02	TCP	541.0	23.65	1.0	US	0

In [ ]:
```
df_mice = df.copy()
```

`IterativeImputer` работает только с числами. Категориальные признаки нужно закодировать.

Поскольку в признаке "country" есть пропуски, то перед кодированием их нужно заменить, например константой 'unknown' — это не «восстановление истины», а способ явно обозначить «неизвестно».

In [ ]:
```
df_mice['country'] = df_mice['country'].fillna('unknown')
```

Кодируем категориальные признаки (One-Hot).

In [ ]:
```
df_encoded = pd.get_dummies(df_mice, columns=['protocol', 'country'])
df_encoded.head()
```

Out[ ]:

	event_time	sensor_id	bytes_sent	duration_sec	failed_logins	is_attack	protocol_ICMP	pr
0	2026-07-01 08:00:00	sensor-01	1458.0	26.77	1.0	1	False	
1	2026-07-01 08:01:00	sensor-03	6334.0	41.74	0.0	0	True	
2	2026-07-01 08:02:00	sensor-02	106.0	43.22	0.0	0	False	
3	2026-07-01 08:03:00	sensor-02	272.0	34.67	0.0	0	False	
4	2026-07-01 08:04:00	sensor-02	541.0	23.65	1.0	0	False	

Формируем полную матрицу признаков для импутации. В нее входят: числовые признаки + закодированные категории.

In [ ]:
```
df_clean = df_encoded.drop(columns=['event_time', 'sensor_id'])
df_clean.head(2)
```

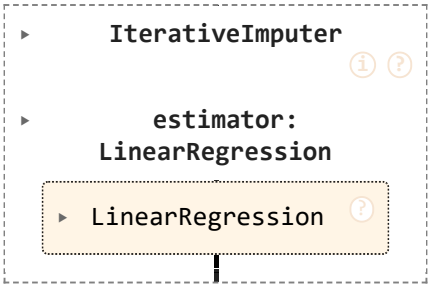
Out[ ]:

	bytes_sent	duration_sec	failed_logins	is_attack	protocol_ICMP	protocol_TCP	protocol_U
0	1458.0	26.77	1.0	1	False	False	Tı
1	6334.0	41.74	0.0	0	True	False	Fa

Настройка параметров класса `IterativeImputer`.

In [ ]:
```
from sklearn.linear_model import LinearRegression
imputer = IterativeImputer(estimator = LinearRegression(),
                           initial_strategy = 'median',
                           max_iter = 10,
                           random_state = 7)
imputer
```

Out[ ]:



Метод `fit()` вычисляет статистику для начальной инициализации пропусков и обучает модель для импутации. В нашем случае - вычисляет коэффициенты уравнения регрессии.

Метод `transform()` использует вычисленные прогнозные значения модели линейной регрессии для замены пропусков.

На выходе получаем исходные данные с заполненными пропусками, представленные в виде двумерного массива.

In [ ]:
```
X_imputed_array = imputer.fit_transform(df_clean)
# Преобразование массива в таблицу
df_mice_imputed = pd.DataFrame(X_imputed_array, columns=df_clean.columns, index=df_en
df_mice_imputed.head(7)
```

Out[ ]:

	bytes_sent	duration_sec	failed_logins	is_attack	protocol_ICMP	protocol_TCP	protocol_
0	1458.000000	26.77	1.0	1.0	0.0	0.0	
1	6334.000000	41.74	0.0	0.0	1.0	0.0	
2	106.000000	43.22	0.0	0.0	0.0	1.0	
3	272.000000	34.67	0.0	0.0	0.0	1.0	
4	541.000000	23.65	1.0	0.0	0.0	1.0	
5	9447.000000	52.33	0.0	0.0	0.0	0.0	
6	3178.309897	25.00	0.0	0.0	0.0	0.0	

Достоинства многомерных методов заполнения пропусков:

1. Учёт взаимосвязей между признаками. В отличие от одномерной импутации (медиана/среднее), многомерная использует контекст: "duration_sec" предсказывается с учётом "protocol", "country", "bytes_sent". Для сетевого трафика это критично: у ICMP типичные длительности совсем не такие, как у HTTP.
2. Гибкость модели внутри импутера. Можно задавать различные регрессоры/классификаторы (BayesianRidge, ExtraTrees, RandomForest) и подбирать под тип данных. Например, деревья лучше справляются с нелинейностями и выбросами, чем линейные модели.
3. Может неплохо справляться с пропусками типа MAR и частично — с MNAR, если механизм пропусков закодирован в признаках (например, для ICMP часто нет duration_sec — это видно по protocol_ICMP=1).
4. Итеративная природа улучшает согласованность. На каждой итерации предсказания уточняются с учётом уже обновлённых значений других признаков. Это снижает внутренние противоречия между восстановленными значениями.

Недостатки многомерных методов заполнения пропусков:

1. Вычислительная сложность. На больших датасетах (сотни тысяч–миллионы строк, десятки–сотни признаков) каждая итерация — это обучение модели на почти всей матрице. Для задач кибербезопасности с потоковыми логами это часто неприемлемо.
2. Чувствительность к выбросам. Если в данных много аномальных значений (например, DDoS-пакеты с гигантским bytes_sent), они сильно искажают предсказания. Даже устойчивые модели (ExtraTrees) могут давать «размытые» оценки.
3. Риск «затирания» информативных пропусков. Если пропуск сам по себе — признак атаки (MNAR), то его восстановление может ухудшить качество детекции.
4. Сложность интерпретации. Трудно объяснить бизнес-пользователю или регулятору: «мы восстановили пропуски моделью, которая сама училась на этих же данных». Это может вызывать вопросы к воспроизводимости и прозрачности.

Заполнение пропусков в целевом признаке

In [ ]:
```
df.head(2)
```

Out[ ]:

	event_time	sensor_id	protocol	bytes_sent	duration_sec	failed_logins	country	is_attack
0	2026-07-01 08:00:00	sensor-01	UDP	1458.0	26.77	1.0	NaN	1
1	2026-07-01 08:01:00	sensor-03	ICMP	6334.0	41.74	0.0	DE	0

Мы рассмотрели технологии обработки пропусков в независимых признаках. Что делать, если пропущены значения целевого признака? Для нашего примера - "is_attack".

В классическом машинном обучении пропуск в целевом признаке — это почти всегда брак данных. Неразмеченный пример. Его удаляют и забывают.Но в кибербезопасности пропуск в целевом признаке — это фундаментальная проблема всей индустрии.

Рассмотрим причины появления пропусков в целевом признаке.

Как формируется размеченный датасет для обучения модели обнаружения атак?

Сценарий 1: Ручная разметка аналитиками SOC. Аналитик смотрит на алерт, расследует инцидент и ставит метку: 1 — это была атака, 0 — это норма (ложное срабатывание). Но аналитиков мало, а событий — миллионы в день. В итоге размечены только те события, которые попали в алерты. А что с остальными 99% трафика? Мы не знаем, были там атаки или нет. Для нашей модели их метки — пропущены.

Сценарий 2: Автоматическая разметка по сигнатурам (IPS, антивирус). Определяеются правила, например «Если сработала сигнатура Snort №1234, то это атака (y = 1). Если не сработала ни одна сигнатура — то норма (y = 0)». Но сигнатуры выявляют только известные атаки. Новые zero-day атаки не ловятся сигнатурами, но они есть. И для таких атак будет выставлена ошибочная метка y = 0 — отсутствие атаки.

Сценарий 3: Исторические инциденты. Исследуются дампы трафика за период, когда, как точно известно, был взлом (по постфактум, из отчета IR-команды). Значения целевого признака за весь этот период помечаются как y = 1. Но на самом деле атака могла начаться раньше или продолжаться позже — мы просто не знаем точных границ. Метки зашумлены.

Получается парадоксальная ситуация: в кибербезопасности в большинстве случаев почти все данные, которые считаются «не-атакой», на самом деле являются неразмеченными.

Классификация подходов к обработке пропусков в целевом признаке.

1. Игнорирование (удаление неразмеченных строк).
2. Слабая супервизия (Weak Supervision). Используются эвристики, правила и косвенные признаки, чтобы автоматически присвоить метки неразмеченным данным. Например: «Если IP из черного списка — y = 1. Если IP из белого списка корпоративных подсетей — y = 0. Остальное — удаляем».
3. Обучение на основе положительных и неразмеченных данных (Positive-Unlabeled Learning - PU Learning).
4. Полу-контролируемое обучение (Semi-Supervised / Self-Training).

Рассмотрим технологии применения этих подходов для работы с пропусками в целевом признаке.

Предположим собраны данные о сетевом трафике за месяц. Наблюдения были размечены аналитиками. При этом часть метки целевого признака (y = 1) были поставлены только для наблюдений, соответствующих атаке (например, Атака = высокое кол-во байт + короткий duration + черный IP). Для остальных наблюдений - неразмеченный трафик (y = NaN).

Примечание. Признак "true_label" в реальных задачах неизвестен. Он используется как "истина" в учебных целях, для возможности расчета метрик качества классификации.

In [ ]:
```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.metrics import classification_report

np.random.seed(42)
```

```
n_samples = 2000

# Генерируем признаки
data = {
    'bytes_sent': np.random.exponential(scale=1000, size=n_samples),
    'duration': np.random.exponential(scale=5, size=n_samples),
    'n_packets': np.random.poisson(lam=20, size=n_samples),
    'is_from_blacklisted_ip': np.random.binomial(1, 0.02, size=n_samples),
    'entropy': np.random.uniform(3, 8, size=n_samples) # Новый признак
}
df = pd.DataFrame(data)

# СЛОЖНЫЕ ПРАВИЛА АТАКИ (нелинейные, с шумом)
# Атака = (высокий трафик + низкая длительность) ИЛИ (черный IP) ИЛИ (высокая энтропи
attack_mask = (
    ((df['bytes_sent'] > 2500) & (df['duration'] < 2)) |
    (df['is_from_blacklisted_ip'] == 1) |
    ((df['entropy'] > 7.0) & (df['n_packets'] > 30))
)
# Добавляем шум: 10% атак "маскируются" под нормальный трафик
noise_mask = np.random.binomial(1, 0.1, size=n_samples).astype(bool)
attack_mask = attack_mask & ~noise_mask

df['true_label'] = attack_mask.astype(int)

# КРИТИЧЕСКИ ВАЖНО: Разметка всего 1% данных (как в реальной жизни)
mask_labeled = np.random.binomial(1, 0.01, size=n_samples).astype(bool)
df['y'] = np.nan
df.loc[mask_labeled, 'y'] = df.loc[mask_labeled, 'true_label']

df.head()
```

Out[ ]:

	bytes_sent	duration	n_packets	is_from_blacklisted_ip	entropy	true_label	y
0	469.268090	1.517064	21	0	3.531942	0	NaN
1	3010.121431	1.418309	24	0	5.475858	1	NaN
2	1316.745694	11.835862	14	0	6.046113	0	NaN
3	912.942554	1.435386	18	0	4.374542	0	NaN
4	169.624870	1.586926	22	0	6.583989	0	NaN

In [ ]:
```
print(f"Всего строк: {len(df)}")
print(f"Истинных атак: {df['true_label'].sum()} ({df['true_label'].mean():.1%})")
print(f"Размечено строк: {df['y'].notna().sum()}")
print(f"Размеченных атак: {df.loc[df['y'].notna(), 'y'].sum()}")
```

```
Всего строк: 2000
Истинных атак: 82 (4.1%)
Размечено строк: 24
Размеченных атак: 2.0
```

Игнорирование (удаление неразмеченных строк).

Для обучения модели используются только те наблюдения, для которых целевой признак точно известен.

In [ ]:
```
# Оставляем только размеченные строки
df_labeled = df.dropna(subset=['y']).copy()
# Формируем таблицу с независимыми признаками
```

```
X = df_labeled.drop(['y', 'true_label'], axis = 1)
# Формируем таблицу с целевым признаком
y = df_labeled[['y']]
display(X.head(), y.head())
```

	bytes_sent	duration	n_packets	is_from_blacklisted_ip	entropy
29	47.563850	5.309106	21	0	6.400793
54	911.054412	9.484779	17	0	4.953587
66	151.898148	1.124249	23	0	7.393488
229	997.334386	1.782797	19	0	6.353800
306	726.291119	0.455605	18	0	6.824239

	y
29	0.0
54	0.0
66	0.0
229	0.0
306	0.0

In [ ]:
```
# Обучаем модель
model_naive = RandomForestClassifier(n_estimators=100, random_state=42)
model_naive.fit(X, y)

# Предсказываем значения целевого признака для всех наблюдений (включая неразмеченные
X_all = df.drop(columns=['y', 'true_label'])
y_pred_naive = model_naive.predict(X_all)

# Предсказанные метки целевого признака
y_pred_naive[:10]
```

Out[ ]:
```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

Оценим какую долю реальных атак (положительных случаев) модель сумела правильно обнаружить.

In [ ]:
```
from sklearn.metrics import recall_score
print("\n=== Наивный подход ===")
print(recall_score(df['true_label'], y_pred_naive))
```

```
=== Наивный подход ===
0.15853658536585366
```

Величина 0.15 означает, что качество восстановления пропусков очень низкое: 85% атак прошла незамеченной. В реальных SOC такую модель использовать нельзя.

Плюс: просто.

Минус: полностью игнорируется значительная часть данных. Модель учится только на «очевидных» случаях.

Допустим, только когда:

- достаточно размеченных данных (10%+), и они репрезентативны;
- атаки простые и хорошо разделимы;
- нужно быстрое baseline-решение «на вчера».

Полу-контролируемое обучение (Semi-Supervised / Self-Training).

Первоначально модель обучается на размеченных данных. Затем она используется для предсказания целевого признака, чтобы разметить неразмеченные данные с высокой уверенностью. После чего модель переобучается на расширенной выборке. Выполняется несколько итераций для получения требуемого качества классификации атак.

Для реализации этого подхода можно использовать класс `SelfTrainingClassifier` из библиотеки `scikit-learn`.

В нем используется следующий алгоритм:

1. Неразмеченные данные маркируются "-1".
2. Отфильтровываются все строки с -1 и обучается базовая модель (DecisionTreeClassifier) только на размеченных данных.
3. Берутся все данные (включая те, что были с -1) и для целевого признака вычисляются предсказания - вероятности принадлжености наблюдения к классу 1 (атака).
4. Для тех предсказаний, где уверенность (вероятность) выше threshold (в нашем случае 0.9), -1 заменяется на на предсказанный класс (0 или 1).
5. Процесс повторяется, пока не закончатся -1 или не будет достигнут лимит итераций (`max_iter=5`).

In [ ]:
```
from sklearn.semi_supervised import SelfTrainingClassifier
from sklearn.tree import DecisionTreeClassifier

# SelfTrainingClassifier требует, чтобы базовая модель умела предсказывать вероятност
base_estimator = DecisionTreeClassifier(random_state = 42)

# Готовим данные: размеченные — с метками, неразмеченные — с меткой -1
X_semi = df.drop(columns=['y', 'true_label'])
y_semi = df['y'].fillna(-1).astype(int)

# threshold=0.9 — добавляем в обучение только те предсказания,
# в которых модель уверена более чем на 90%
self_trainer = SelfTrainingClassifier(base_estimator, threshold=0.9, max_iter=5)
self_trainer.fit(X_semi, y_semi)

y_pred_semi = self_trainer.predict(X_semi)

print("\n=== Self-Training (полу-контролируемое обучение) ===")
print(recall_score(df['true_label'], y_pred_semi))
```

```
=== Self-Training (полу-контролируемое обучение) ===
0.36585365853658536
```

Обучение на основе положительных и неразмеченных данных (Positive-Unlabeled Learning - PU Learning).

Явно признается наличие только «положительных» примеров (атаки) и (которые могут быть как атаками, так и нет). Используются специальные алгоритмы, которые умеют учиться в таких условиях.

Явно признается наличие только «положительных» примеров P (positive — точно атаки) и «неразмеченных» примеров U (unlabeled — неизвестно что). Используется двухшаговый алгоритм.

Качество восстановления пропусков остается достаточно низким.

Self-Training имеет преимущества перед наивным подходом когда:

- размеченных данных немного больше (5-15%);
- атаки относительно однородны, и модель может уверенно предсказывать вероятности;
- есть возможность итеративно уточнять модель и валидировать её на свежих размеченных данных.

Идея алгоритма (упрощенный вариант):

1. Считаем все unlabeled примеры потенциально «не-атакой».
2. Однако среди них точно есть атаки. Чтобы их найти, используется аномальное обнаружение (Isolation Forest) на размеченных «положительных» примерах. То, что похоже на известные атаки — помечаем как «вероятные атаки».
3. Остальные unlabeled, которые выглядят «нормально», с высокой вероятностью действительно не являются атаками. Они помечаются как «вероятное y = 0».
4. Обучается финальный классификатор на расширенной выборке.

In [ ]:
```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.metrics import classification_report

X_features = df.drop(columns=['y', 'true_label'])
X_positive = X_features[df['y'] == 1]           # Точно атаки (P)
X_unlabeled = X_features[df['y'].isna()]        # Неизвестно (U)

# ШАГ 1: Обучаем Isolation Forest на НЕРАЗМЕЧЕННЫХ данных.
# Мы предполагаем, что 90-95% unlabeled - это нормальный трафик.
# Параметр contamination=0.1 означает, что мы ожидаем найти около 10% аномалий (скрыт
iso_forest = IsolationForest(contamination=0.1, random_state=42)
iso_forest.fit(X_unlabeled)

# ШАГ 2: Предсказываем метки для unlabeled.
# predict возвращает 1 для inliers (норма) и -1 для outliers (аномалии/вероятные атак
pred_unlabeled = iso_forest.predict(X_unlabeled)

# Преобразуем: outliers (-1) -> y=1 (атака), inliers (1) -> y=0 (benign)
y_unlabeled_pu = np.where(pred_unlabeled == -1, 1, 0)

# ШАГ 3: Формируем расширенную выборку
X_pu = pd.concat([X_positive, X_unlabeled], ignore_index=True)
y_pu = np.concatenate([
    np.ones(len(X_positive)),  # Известные атаки -> y=1
    y_unlabeled_pu             # Расширенные метки для unlabeled
])

# ШАГ 4: Обучаем финальный классификатор
model_pu = RandomForestClassifier(n_estimators=200, random_state=42)
model_pu.fit(X_pu, y_pu)

y_pred_pu = model_pu.predict(X_features)
```

```
print("\n=== PU Learning ===")
print(recall_score(df['true_label'], y_pred_pu))
```

```
=== PU Learning ===
0.6097560975609756
```

PU Learning дает преимущество, когда:

- разметки критически мало (< 2-5% от общего объема данных);
- атаки разнообразны и сложны, и размеченная выборка не покрывает все их паттерны;
- стоимость пропущенной атаки (False Negative) крайне высока (например, утечка данных).

Литература

1. Криволапов, С. Я. Анализ данных. Методы теории вероятностей и математической статистики на языке Python : учебное пособие / С.Я. Криволапов. – Москва : ИНФРА-М, 2025. – 678 с.
2. Груздев А.В. Предварительная подготовка данных в Python. Том 2: План, примеры и метрики качества. – М.: ДМК Пресс, 2023. – 814 с.
3. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2022 — Часть 1 — 2022. — 107 с.
4. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2023 — Часть 2 — 2023. — 92 с.